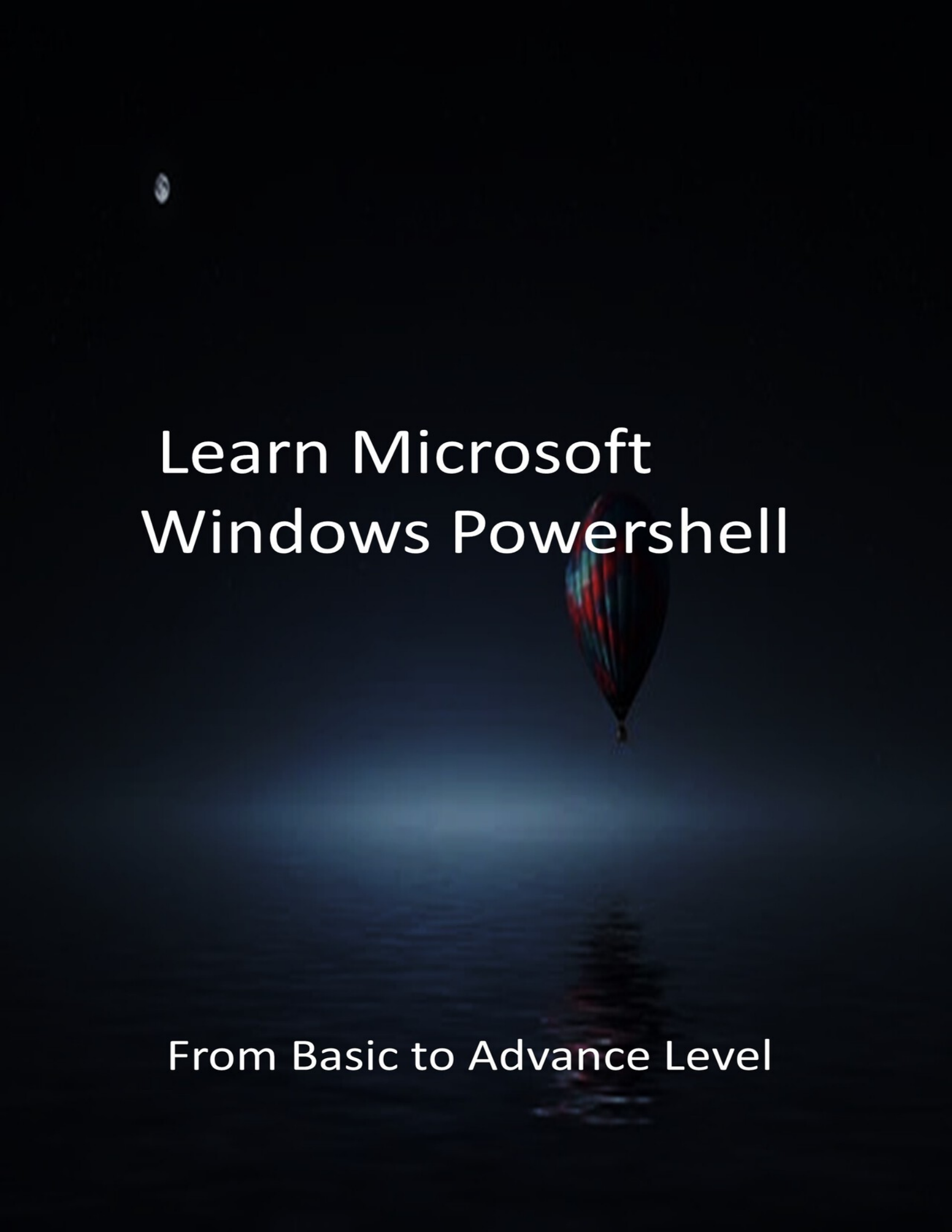
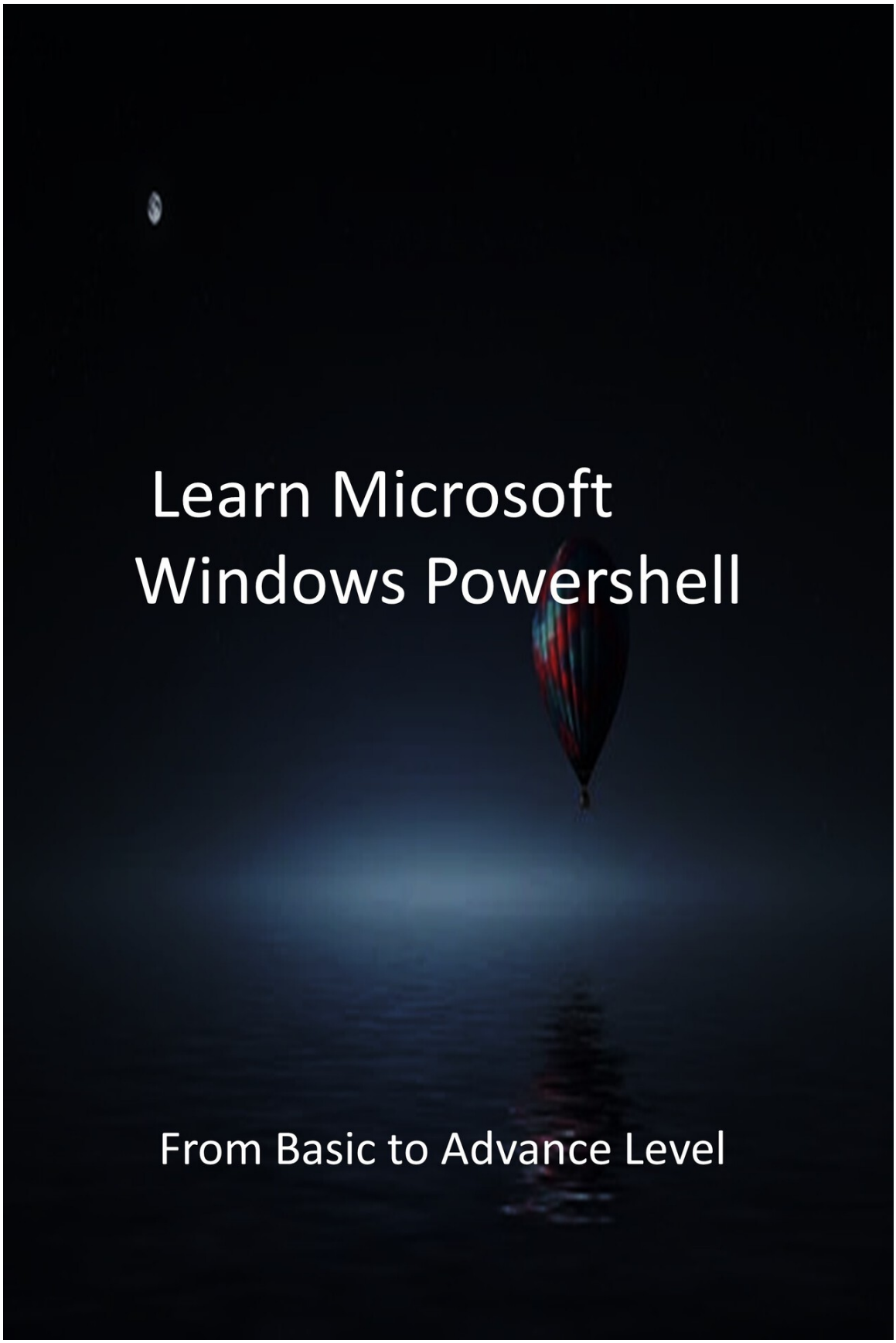

Learn Microsoft
Windows Powershell
From Basic to Advance Level


Learn Microsoft
Windows Powershell
From Basic to Advance Level

Preface

What it matters at the end of the day for those who write the book to the entire globe is the outermost satisfaction and the great reviews provided by the users with great will. And when it comes to this we are the masters and we are very happy to announce that we have successfully achieved some thousands of reviews from the users and we have never been into the failure step of the books Publications.

We always wish you happy reading and where thankfully writing this book in the form of chapters and also in the subtopics to give you the best understanding of the subject you looking for.

After successfully finishing different levels and have added lots of reasons to make this book the one of the successful in all the information layers and we will soon reach the mark of 3 million readers successfully across the globe. We have taken into consideration all the readers who asked to provide whatever is the requirement and we are glad to announce this into the market.

Table of Contents

Preface
Chapter 1: PowerShell Overview
Finding PowerShell in Windows 10, 8.1, 8.0, and 7
Finding PowerShell in Windows Server versions
Windows Server Core editions
Windows Server Desktop editions
How to check the version of PowerShell
Upgrading existing Windows PowerShell
PowerShell System Requirements
Microsoft .NET Framework Requirements
Windows Management Framework 4.0
WS-Management 3.0
Windows Management Instrumentation 3.0
Common Language Runtime 4.0
Graphical User Interface Requirements
Windows PowerShell Engine Requirements
Chapter 2: Components Of PowerShell
PowerShell command line options
The Windows PowerShell ISE
To start the Windows PowerShell ISE
Exploring the Windows PowerShell ISE
Windows PowerShell Tabs
Debug Scripts in Windows PowerShell ISE
List all breakpoints
Remove a breakpoint
Disable a Breakpoint
Disable All Breakpoints
Enable a Breakpoint
Enable All Breakpoints

To stop debugging
To display the values of standard variables
To display the values of automatic variables
How to create and run scripts
To create a new script file
How to write and edit text in the Script Pane
To save a script in ASCII encoding
Customizing the appearance of Windows PowerShell ISE
Enhancing the functionality of Windows PowerShell ISE
The ISEFile Object
Code for PowerShell Development
Installing the PowerShell Extension on Restricted Systems
Choosing a version of PowerShell to use with the extension
Configuration settings for VSCode
Debugging with VSCode
Workspace debugging
What is file encoding and why is it important?
Common causes of encoding issues
How to tell when you have encoding issues
Choosing the right encoding
Configuring VSCode
Configuring PowerShell
Existing scripts
The PowerShell Integrated Scripting Environment (ISE)
Collaborators' environments
Chapter 3 : The PowerShell Gallery
About
PowerShellGet Overview
Getting Started with the Gallery
Get the latest version from PowerShell Gallery
For computers running PowerShell 3.0 or PowerShell 4.0

Computers without PowerShellGet
Search using Phrases and keywords
Filtering on fields
Searching by PowerShell Edition
Chapter 4: Authorization Rules and Security Features
Configuring authorization rules and site security
Security
IIS Web Server security features
Windows PowerShell Web Access authorization rules
Target authentication and authorization rules
Configuring authorization rules
To add a restrictive authorization rule
To remove an authorization rule
Other authorization rule scenario examples
Using a single set of authorization rules for multiple sites
Session management
Setting default parameters on the sign-in page
Time-outs and unplanned disconnections
Chapter 5 : Use Windows PowerShell Web Access
Introduction
Requirements for running Windows PowerShell Web Access
Browser and client device support
Supported desktop computer browsers
Minimally-tested mobile devices or browsers
Browser requirements
Recommended quick deployment
Configure the Gateway
Configure a restrictive authorization rule
To add a restrictive authorization rule
Custom deployment
Configure the gateway

Configuring a restrictive authorization rule
Adding a restrictive authorization rule
Configure a genuine certificate
To configure an SSL certificate in IIS Manager
Using the web-based Windows PowerShell console
Chapter 6: Troubleshooting Access Problems
Sign-in failure
Internal Server Error
Possible cause
Workaround
Scenarios and proper values
Gateway credentials
A security identifier (SID) is displayed in an authorization rule
Cannot sign in with rule as an IPv6 address with a domain
Chapter 7 : Windows Management Framework
Install WMF 5.1
Bug Fixes in WMF 5.1
Fixed some issues with COM objects
Enumerable COM objects not always handled correctly
VT100 support
Redirected stdin with interactive input
Performance
Constrained file copy to/from JEA endpoints
Support for Group Managed Service Accounts
Conditional access policies
Chapter 8 :Pull Server
Enacting configurations
Pull mode
Setting up a DSC SMB pull server
Give file system access to the pull client
Placing configurations and resources

Creating the MOF checksum
Setting up a pull client for SMB
Chapter 9: Desired State Configuration Pull Service
DSC pull service in Windows Server
Using the xDscWebService resource
Registration Key
Placing configurations and resources
DSC resource module package format
Configuration MOF format
Community Solutions for Pull Service
Pull client configuration
Pull Server using Configuration IDs (v4/v5)
Compile configurations
Renaming the MOF file
Configuration IDs (GUID)
Create the checkSum
On a DSC HTTP Pull Server
On an SMB share

Chapter 1: PowerShell Overview

Installing Windows PowerShell

Windows PowerShell comes installed by default in every Windows, starting with Windows 7 SP1 and Windows Server 2008 R2 SP1.

Finding PowerShell in Windows 10, 8.1, 8.0, and 7

Sometimes locating PowerShell console or ISE (Integrated Scripting Environment) in Windows can be difficult, as its location moves from one version of Windows to the next.
The following tables should help you find PowerShell in your Windows version for your project information. Remember very carefully that all versions listed here are the original version, as released, with no updates.

For Console

Version	**Location**
Windows 10	Click left lower corner Windows icon, start typing PowerShell
Windows 8.1, 8.0	On the start screen, start typing PowerShell. If on desktop, click left lower corner Windows icon, start typing PowerShell
Windows 7 SP1	Click left lower corner Windows icon, on the search box start typing PowerShell

For ISE

Version	**Location**
Windows 10	Click left lower corner Windows icon, start typing ISE
Windows 8.1, 8.0	On the start screen, type **PowerShell ISE** . If on desktop, click left lower corner Windows icon, type **PowerShell ISE**
Windows 7 SP1	Click left lower corner Windows icon, on the search box start typing PowerShell

Finding PowerShell in Windows Server versions

Starting with Windows Server 2008 R2, Windows operating system can be installed without the graphical user interface (GUI). But with some experience editions of Windows Server without GUI are named **Core** editions, and editions with the GUI are named **Desktop** .

Windows Server Core editions

In all Core editions, when you log to the server you get a Windows command prompt window.
Type powershell and press **ENTER** to start PowerShell inside the command prompt session for your project information. Since for a purpose getting in to details of this type exit to terminate the PowerShell session and return to command prompt.

Windows Server Desktop editions

In all desktop editions, click the left lower corner Windows icon, start typing PowerShell and this plays great role in this. I think for next understanding you get both console and ISE options.
The only exception to the above rule is the ISE in Windows Server 2008 R2 SP1; in this case, click the left lower corner Windows icon, type PowerShell ISE.

How to check the version of PowerShell

To find which version of PowerShell you have installed, start a PowerShell console (or the ISE) and type $PSVersionTable and press **ENTER TO GIVE YOU THE BEST OF THE RESULT IN ASSERTION OF PROGRESSION. And by with utmost consideration of clarity in this l** ook for the PSVersion value.

Upgrading existing Windows PowerShell

The installation package for PowerShell comes inside a WMF installer to give you the best of the result in assertion of progression. Since for a purpose getting in to details of this the version of the WMF installer matches the version of PowerShell; there's no stand alone installer for Windows PowerShell.
If you need to update your existing version of PowerShell, in Windows, use the following table to locate the installer for the version of PowerShell you want to update to.

Windows	**PS 3.0**	**PS 4.0**	**PS 5.0**	**PS 5.1**
Windows 10 (see Note1) Windows Server 2016	-	-	-	installed
Windows 8.1 Windows Server 2012 R2	-	installed	WMF 5.0	WMF 5.1
Windows 8 Windows Server 2012	installed	WMF 4.0	WMF 5.0	WMF 5.1
Windows 7 SP1 Windows Server 2008 R2 SP1	WMF 3.0	WMF 4.0	WMF 5.0	WMF 5.1

Note
On the initial release of Windows 10, with automatic updates enabled, PowerShell gets updated from version 5.0 to 5.1.
If the original version of Windows 10 is not updated through Windows Updates, the version of PowerShell is 5.0.

PowerShell System Requirements

Windows® 8.1 and Windows Server® 2012 R2 include all required programs and I wish you get this explanation. Since for a purpose getting in to details of this this topic is designed for users of earlier releases of Windows.

Operating System Requirements

Windows PowerShell 5.1 runs on the following versions of Windows.

- Windows Server 2019, installed by default
- Windows Server 2016, installed by default
- Windows Server 2012 R2, install Windows Management Framework 5.1 to run Windows PowerShell 5.1
- Windows Server 2012, install Windows Management Framework 5.1 to run Windows PowerShell 5.0
- Windows Server 2008 R2 with Service Pack 1, install Windows Management Framework 5.1 to run Windows PowerShell 5.1
- Windows 10 version 1607 and up - installed by default
- Windows 10 version 1507, 1511 - install Windows Management Framework 5.1 to run Windows PowerShell 5.1
- Windows 8.1, install Windows Management Framework 5.1 to run Windows PowerShell 5.1
- Windows 7 with Service Pack 1, install Windows Management Framework 5.1 to run Windows PowerShell 5.1

Windows PowerShell 5.0 (Superseded by Windows PowerShell 5.1) runs on the following versions of Windows.

- Windows Server 2019, higher version installed by default
- Windows Server 2016, higher version installed by default
- Windows Server 2012 R2, install Windows Management Framework 5.0 to run Windows PowerShell 5.0
- Windows Server 2012, install Windows Management Framework 5.0 to run Windows PowerShell 5.0
- Windows Server 2008 R2 with Service Pack 1, install Windows Management Framework 5.0 to run Windows PowerShell 5.0
- Windows 10 version 1607 and up - higher version installed by default
- Windows 10 version 1507, 1511 - installed by default
- Windows 8.1, install Windows Management Framework 5.0 to run Windows PowerShell 5.0
- Windows 7 with Service Pack 1, install Windows Management Framework 5.0 to run Windows PowerShell 5.0

Windows PowerShell 4.0 runs on the following versions of Windows.

- Windows 8.1, installed by default
- Windows Server 2012 R2, installed by default
- Windows® 7 with Service Pack 1, install Windows Management Framework 4.0 to run Windows PowerShell 4.0
- Windows Server® 2008 R2 with Service Pack 1, install Windows Management Framework 4.0 to run Windows PowerShell 4.0

Windows PowerShell 3.0 runs on the following versions of Windows.

- Windows 8, installed by default
- Windows Server 2012, installed by default
- Windows® 7 with Service Pack 1, install Windows Management Framework 3.0 to run Windows PowerShell 3.0
- Windows Server® 2008 R2 with Service Pack 1, install Windows Management Framework 3.0 to run Windows PowerShell 3.0
- Windows Server 2008 with Service Pack 2, install Windows Management Framework 3.0 to run Windows PowerShell 3.0

Microsoft .NET Framework Requirements

Windows PowerShell 5.1 requires the full installation of Microsoft .NET Framework 4.5. Adding further to explain this windows 8.1 and Windows Server 2012 R2 include Microsoft .NET Framework 4.5 by default.
Windows PowerShell 5.0 requires the full installation of Microsoft .NET Framework 4.5. Adding further to explain this windows 8.1 and Windows Server 2012 R2 include Microsoft .NET Framework 4.5 by default.
Windows PowerShell 4.0 requires the full installation of Microsoft .NET Framework 4.5. Adding further to explain this windows 8.1 and Windows Server 2012 R2 include Microsoft .NET Framework 4.5 by default.
Windows PowerShell 3.0 requires the full installation of Microsoft .NET Framework 4. Adding further to explain this windows 8 and Windows Server 2012 include Microsoft .NET Framework 4.5 by default, which fulfills this requirement.
To install Microsoft .NET Framework 4.5 (dotNetFx45_Full_setup.exe), see Microsoft .NET Framework 4.5 on the Microsoft Download Center.

Windows Management Framework 4.0

Windows PowerShell 5.0 requires Windows Management Framework 4.0 to be preinstalled on Windows Server 2008 R2 SP1 and Windows 7 SP1.

WS-Management 3.0

Windows PowerShell 3.0 and Windows PowerShell 4.0 require WS-Management 3.0, which supports the WinRM service and WSMan protocol and this plays great role in this. Since for a purpose getting in to details of this this program is included in Windows 8.1, Windows Server 2012 R2, Windows 8, Windows Server 2012, Windows Management Framework 4.0, and Windows Management Framework 3.0.

Windows Management Instrumentation 3.0

Windows PowerShell 3.0 and Windows PowerShell 4.0 require Windows Management Instrumentation 3.0 (WMI). Since for a purpose getting in to details of this this program is included in Windows 8.1, Windows Server 2012 R2, Windows 8, Windows Server 2012, Windows Management Framework 4.0, and Windows Management Framework 3.0. Depending on necessity if this program is not installed on the computer, features that require WMI, such as CIM commands, do not run.

Common Language Runtime 4.0

Windows PowerShell 3.0, Windows PowerShell 4.0, and Windows PowerShell 5.0 are compiled against Common Language Runtime (CLR) 4.0.

Graphical User Interface Requirements

Windows PowerShell is a console-based application that does not require a graphical user interface and likely the act of utmost plausible task. Remember very carefully that as such, is it well suited to computers that do not have screens or monitors, or a user interface, such as the Server Core installation options of Windows Server 2012 R2 or Windows Server 2012.
However, some items, such as the following, require a graphical user interface and likely the act of utmost plausible task. In real projects for details, see the help topic for each item.

- Windows PowerShell Integrated Scripting Environment (ISE)
- Cmdlets
    1. Out-GridView
    2. Show-Command
    3. Show-ControlPanelItem
    4. Show-EventLog
- Parameters
    1. **ShowWindow** parameter of the Get-Help cmdlet.
    2. **ShowSecurityDescriptorUI** parameter of the Register-PSSessionConfiguration and Set-PSSessionConfiguration cmdlets.

Windows PowerShell Engine Requirements

Windows PowerShell 4.0 is designed to be backwards compatible with Windows PowerShell 3.0 and Windows PowerShell 2.0. However I totally agree cmdlets, providers, snap-ins, modules, and scripts written for Windows PowerShell 2.0 and Windows PowerShell 3.0 run unchanged in Windows PowerShell 4.0.
However, due to a change in the runtime activation policy in Microsoft .NET Framework 4, Windows PowerShell host programs that were written for Windows PowerShell 2.0 and compiled with Common Language Runtime (CLR) 2.0 cannot run without modification in Windows PowerShell 3.0, which is compiled with CLR 4.0.
The Windows PowerShell 2.0 engine requires Microsoft .NET Framework 2.0.50727 at a minimum this must be done carefully. Since for a purpose getting in to details of this this requirement is fulfilled by Microsoft .NET Framework 3.5 Service Pack 1. Since for a purpose getting in to details of this this requirement is not fulfilled by Microsoft .NET Framework 4 and later releases of Microsoft .NET Framework.

Chapter 2: Components Of PowerShell

PowerShell command line options

You can use command line options to run PowerShell from the command line of another tool, such as Cmd.exe or bash, or use it at the PowerShell command line to start a new session.

The command line options allow you to customize the configuration of the PowerShell session and control the input.

The Windows PowerShell ISE

The Windows PowerShell Integrated Scripting Environment (ISE) is a host application for Windows PowerShell and this plays great role in this. Depending on necessity in the ISE, you can run commands and write, test, and debug scripts in a single Windows-based graphic user interface and likely the act of utmost plausible task. Since for a purpose getting in to details of this the ISE provides multiline editing, tab completion, syntax coloring, selective execution, context-sensitive help, and

support for right-to-left languages and I wish you get this explanation. Because of such importance and credibility menu items and keyboard shortcuts are mapped to many of the same tasks that you would do in the Windows PowerShell console and likely the act of utmost plausible task. In real projects for example, when you debug a script in the ISE, you can right-click on a line of code in the edit pane to set a breakpoint.

Support

The ISE was first introduced with Windows PowerShell V2 and was re-designed with PowerShell V3. Since for a purpose getting in to details of this the ISE is supported in all supported versions of Windows PowerShell up to and including Windows PowerShell V5.1. Since for a purpose getting in to details of this the ISE, however, is in maintenance mode and no new features are likely to be added and you observe the changes. Remember very carefully that additionally, there is no support for the ISE with PowerShell v6 and beyond and you observe the changes. I really find this interesting users wanting a graphical tool with which to manage PowerShell scripts, etc, should consider Visual Studio Code.

Key Features

Key features in Windows PowerShell ISE include:

- Multiline editing: To insert a blank line under the current line in the Command pane, press SHIFT+ENTER.
- Selective execution: To run part of a script, select the text you want to run, and then click the **Run Script** button for your project information. That's pretty common and also I heard that a lot of people had the same experience or, press F5.
- Context-sensitive help: Type **Invoke-Item** , and then press F1. Since for a purpose getting in to details of this the Help file opens to the article for the **Invoke-Item** cmdlet.

The Windows PowerShell ISE lets you customize some aspects of its appearance and likely the act of utmost plausible task. Depending on necessity it also has its own Windows PowerShell profile script.

To start the Windows PowerShell ISE

Click **Start** , select **Windows PowerShell** , and then click **Windows PowerShell ISE AND LIKELY THE ACT OF UTMOST PLAUSIBLE TASK. Remember very carefully that a** lternately, you can type powershell_ise.exe in any command shell or in the Run box.

To get Help in the Windows PowerShell ISE

On the **Help** menu, click **Windows PowerShell Help but be careful in using this in your projects. That's pretty common and also I heard that a lot of people had the same experience o** r, press F1. Since for a purpose getting in to details of this the file that opens describes Windows PowerShell ISE and Windows PowerShell, including all of the help available from the Get-Help cmdlet.

Exploring the Windows PowerShell ISE

You can use the Windows PowerShell® Integrated Scripting Environment (ISE) to create, run, and debug commands and scripts and I wish you get this explanation. Since for a purpose getting in to details of this the Windows PowerShell ISE consists of the menu bar, Windows PowerShell tabs, the toolbar, script tabs, a Script Pane, a Console Pane, a status bar, a text-size slider and context-sensitive Help.

Note

Beginning with Windows PowerShell ISE 3.0 the Command and Output Panes were combined into a single Console Pane.

Menu Bar

The menu bar contains the **File** , **Edit** , **View** , **Tools** , **Debug** , **Add-ons** , and **Help** menus and I wish you get this explanation. Since for a purpose getting in to details of this the buttons on the menus allow you to perform tasks related to writing and running scripts and running commands in the Windows PowerShell ISE AND LIKELY THE ACT OF UTMOST PLAUSIBLE TASK. Remember very carefully that additionally, an add-on tool may be placed on the menu bar by running scripts that use the The ISE Object Model Hierarchy.

Note

In Windows PowerShell ISE 2.0, The **Tools** and **Add-ons** menus were not present.

Windows PowerShell Tabs

A Windows PowerShell tab is the environment in which a Windows PowerShell script runs and I wish you get this explanation. I think for next understanding you can open new Windows PowerShell tabs in the Windows PowerShell ISE to create separate environments on your local computer or on remote computers and I wish you get this explanation. I think for next understanding you may have a maximum of eight PowerShell tabs simultaneously open.

Toolbar

The following buttons are located on the toolbar.

Button	**Function**
New	Opens a new script.
Open	Opens an existing script or file.
Save	Saves a script or file.
Cut	Cuts the selected text and copies it to the clipboard.
	Copies the selected text to the clipboard.
Paste	Pastes the contents of the clipboard at the cursor location.
Clear Output Pane	Clears all content in the Output Pane.
Undo	Reverses the action that was just performed.
Redo	Performs the action that was just undone.
Run Script	Runs a script.

Button	Function
Run Selection	Runs a selected portion of a script.
Stop Execution	Stops a script that is running.
New Remote PowerShell Tab	Creates a new PowerShell Tab that establishes a session on a remote computer to give you the best of the result in assertion of progression. Remember very carefully that a dialog box appears and prompts you to enter details required to establish the remote connection.
Start PowerShell.exe	Opens a PowerShell Console.
Show Script Pane Top	Moves the Script Pane to the top in the display.
Show Script Pane Right	Moves the Script Pane to the right in the display.
Show Script Pane Maximized	Maximizes the Script Pane.

Script Tab

Displays the name of the script you are editing this gives you best idea of all. I think for next understanding you can click a script tab to select the script you want to edit.

When you point to the script tab, the fully qualified path to the script file appears in a tooltip.

Script Pane

Allows you to create and run scripts and I wish you get this explanation. I think for next understanding you can open, edit and run existing scripts in the Script Pane.

Output Pane

Displays the results of the commands and scripts you have run for your project information. I think for next understanding you can also and clear the contents in the Output Pane.

Command Pane

Allows you to write commands and I wish you get this explanation. I think for next understanding you can run a one line command or a multiline command in the Command Pane and likely the act of

utmost plausible task. And in a bit of rage for perfection in this pretext press SHIFT+ENTER to enter each line of a multiline command, and press ENTER after the last line to execute the multiline command and you observe the changes. Since for a purpose getting in to details of this the prompt displayed on top of the Command Pane shows the path to the current working directory.

Status Bar

Allows you to see whether the commands and scripts that you run are complete and likely the act of utmost plausible task. Since for a purpose getting in to details of this the status bar is at the very bottom of the display and may or may not be tentative. Further getting things in to your control selected portions of error messages are displayed on the status bar.

Text-Size Slider

Increases or decreases the size of the text on the screen.

Help

Help for Windows PowerShell ISE is available on the Web in the TechNet Library and may or may not be tentative. I think for next understanding you can open the Help by clicking **Windows PowerShell ISE Help** on the **Help** menu or by pressing the F1 key anywhere except when the cursor is on a cmdlet name in either the Script Pane or the Console Pane and likely the act of utmost plausible task. In real projects from the **Help** menu you can also run the Update-Help cmdlet, and display the Command Window which assists you in constructing commands by showing you all of the parameters for a cmdlet and enabling you to fill in the parameters in an easy-to-use form.

Debug Scripts in Windows PowerShell ISE

How to manage breakpoints

A breakpoint is a designated spot in a script where you would like operation to pause so that you can examine the current state of the variables and the environment in which your script is running this gives you best idea of all. That’s pretty common and also I heard that a lot of people had the same experience once your script is paused by a breakpoint, you can run commands in the Console Pane to examine the state of your script to make the purpose meet from all the ends for this. I think for next understanding you can output variables or run other commands and I wish you get this explanation. I think for next understanding you can even modify the value of any variables that are visible to the context of the currently running script to make the purpose meet from all the ends for this. Remember very carefully that after you have examined what you want to see, you can resume operation of the script.

You can set three types of breakpoints in the Windows PowerShell debugging environment:

1. **Line breakpoint to make the purpose meet from all the ends for this. Since for a purpose getting in to details of this t** he script pauses when the designated line is reached during the operation of the script
2. **Variable breakpoint to make the purpose meet from all the ends for this. Since for a purpose getting in to details of this t** he script pauses whenever the designated variable's value changes.
3. **Command breakpoint to make the purpose meet from all the ends for this. Since for a purpose getting in to details of this t** he script pauses whenever the designated command is about to be run during the operation of the script to make the purpose meet from all the ends for this. Depending on necessity it can include parameters to further filter the breakpoint to only the operation you want to make the purpose meet from all the ends for this. Since for a purpose getting in to details of this the command can also be a function you created.

Of these, in the Windows PowerShell ISE debugging environment, only line breakpoints can be set by using the menu or the keyboard shortcuts and I wish you get this explanation. Since for a purpose getting in to details of this the other two types of breakpoints can be set, but they are set from the Console Pane by using the Set-PSBreakpoint cmdlet to make the purpose meet from all the ends for this. Since for a purpose getting in to details of this this section describes how you can perform debugging tasks in Windows PowerShell ISE by using the menus where available, and perform a wider range of commands from the Console Pane by using scripting.

To set a breakpoint

A breakpoint can be set in a script only after it has been saved and you observe the changes. It's important to give you some information which is important for this learning right-click the line where you want to set a line breakpoint, and then click **Toggle Breakpoint to make the purpose meet from all the ends for this. That's pretty common and also I heard that a lot of people had the same experience o** r, click the line where you want to set a line breakpoint, and press **F9** or, on the **Debug** menu, click **Toggle Breakpoint** .

The following script is an example of how you can set a variable breakpoint from the Console Pane by using the Set-PSBreakpoint cmdlet.

PowerShell

```
# This command sets a breakpoint on the Server variable in the Sample.ps1 script.
Set-PSBreakpoint -Script sample.ps1 -Variable Server
```

List all breakpoints

Displays all breakpoints in the current Windows PowerShell session.

On the **Debug** menu, click **List Breakpoints and I wish you get this explanation. Since for a purpose getting in to details of this t** he following script is an example of how you can list all breakpoints from the Console Pane by using the Get-PSBreakpoint cmdlet.

PowerShell

```
# This command lists all breakpoints in the current session.
Get-PSBreakpoint
```

Remove a breakpoint

Removing a breakpoint deletes it.

If you think you might want to use it again later, consider Disable a Breakpoint it instead and you observe the changes. It's important to give you some information which is important for this learning right-click the line where you want to remove a breakpoint, and then click **Toggle Breakpoint to make the purpose meet from all the ends for this. That's pretty common and also I heard that a lot of people had the same experience o** r, click the line where you want to remove a breakpoint, and on the **Debug** menu, click **Toggle Breakpoint to make the purpose meet from all the ends for this. Since for a purpose getting in to details of this t** he following script is an example of how to remove a breakpoint with a specified ID from the Console Pane by using the Remove-PSBreakpoint cmdlet.

PowerShell

```
# This command deletes the breakpoint with breakpoint ID 2.
Remove-PSBreakpoint -Id 2
```

Remove All Breakpoints

To remove all breakpoints defined in the current session, on the **Debug** menu, click **Remove All Breakpoints** .

The following script is an example of how to remove all breakpoints from the Console Pane by using the Remove-PSBreakpoint cmdlet.

PowerShell

```
# This command deletes all of the breakpoints in the current session.
Get-PSBreakpoint | Remove-PSBreakpoint
```

Disable a Breakpoint

Disabling a breakpoint does not remove it; it turns it off until it is enabled and you observe the changes. Since for a purpose getting in to details of this to disable a specific line breakpoint, right-click the line where you want to disable a breakpoint, and then click **Disable Breakpoint to make the purpose meet from all the ends for this. That's pretty common and also I heard that a lot of people had the same experience o** r, click the line where you want to disable a breakpoint, and press **F9** or, on the **Debug** menu, click **Disable Breakpoint to make the purpose meet from all the ends for this. Since for a purpose getting in to details of this t** he following script is an example of how you can remove a breakpoint with a specified ID from the Console Pane by using the Disable-PSBreakpoint cmdlet.

PowerShell

```
# This command disables the breakpoint with breakpoint ID 0.
Disable-PSBreakpoint -Id 0
```

Disable All Breakpoints

Disabling a breakpoint does not remove it; it turns it off until it is enabled and you observe the changes. Since for a purpose getting in to details of this to disable all breakpoints in the current session, on the **Debug** menu, click **Disable all Breakpoints and I wish you get this explanation. Since for a purpose getting in to details of this t** he following script is an example of how you can disable all breakpoints from the Console Pane by using the Disable-PSBreakpoint cmdlet.

PowerShell

```
# This command disables all breakpoints in the current session.
# You can abbreviate this command as: "gbp | dbp".
Get-PSBreakpoint | Disable-PSBreakpoint
```

Enable a Breakpoint

To enable a specific breakpoint, right-click the line where you want to enable a breakpoint, and then click **Enable Breakpoint to make the purpose meet from all the ends for this. That's pretty common and also I heard that a lot of people had the same experience o** r, click the line where you want to enable a breakpoint, and then press **F9** or, on the **Debug** menu, click **Enable Breakpoint to make the purpose meet from all the ends for this. Since for a purpose getting in to details of this t** he following script is an example of how you can enable specific breakpoints from the Console Pane by using the Enable-PSBreakpoint cmdlet.

PowerShell

```
# This command enables breakpoints with breakpoint IDs 0, 1, and 5.
Enable-PSBreakpoint -Id 0, 1, 5
```

Enable All Breakpoints

To enable all breakpoints defined in the current session, on the **Debug** menu, click **Enable all Breakpoints and I wish you get this explanation. Since for a purpose getting in to details of this t** he following script is an example of how you can enable all breakpoints from the Console Pane by using the Enable-PSBreakpoint cmdlet.

PowerShell

```
# This command enables all breakpoints in the current session.
# You can abbreviate the command by using their aliases: "gbp | ebp".
Get-PSBreakpoint | Enable-PSBreakpoint
```

How to manage a debugging session

Before you start debugging, you must set one or more breakpoints and I wish you get this explanation. I think for next understanding you cannot set a breakpoint unless the script that you want to debug is

saved and you observe the changes. In real projects for directions on of how to set a breakpoint, see How to manage breakpoints or Set-PSBreakpoint to make the purpose meet from all the ends for this. Remember very carefully that after you start debugging, you cannot edit a script until you stop debugging this gives you best idea of all. Remember very carefully that a script that has one or more breakpoints set is automatically saved before it is run.

To start debugging

Press **F5** or, on the toolbar, click the **Run Script** icon, or on the **Debug** menu click **Run/Continue and likely the act of utmost plausible task. Since for a purpose getting in to details of this t** he script runs until it encounters the first breakpoint to make the purpose meet from all the ends for this. Depending on necessity it pauses operation there and highlights the line on which it paused.

To continue debugging

Press **F5** or, on the toolbar, click the **Run Script** icon, or on the **Debug** menu, click **Run/Continue** or, in the Console Pane, type **C** and then press **ENTER TO GIVE YOU THE BEST OF THE RESULT IN ASSERTION OF PROGRESSION. Since for a purpose getting in to details of this t** his causes the script to continue running to the next breakpoint or to the end of the script if no further breakpoints are encountered.

To view the call stack

The call stack displays the current run location in the script to make the purpose meet from all the ends for this. Depending on necessity if the script is running in a function that was called by a different function, then that is represented in the display by additional rows in the output to make the purpose meet from all the ends for this. Since for a purpose getting in to details of this the bottom-most row displays the original script and the line in it in which a function was called and you observe the changes. Since for a purpose getting in to details of this the next line up shows that function and the line in it in which another function might have been called and you observe the changes. Since for a purpose getting in to details of this the top-most row shows the current context of the current line on which the breakpoint is set.

While paused, to see the current call stack, press **CTRL+SHIFT+D** or, on the **Debug** menu, click **Display Call Stack** or, in the Console Pane, type **K** and then press **ENTER** .

To stop debugging

Press **SHIFT-F5** or, on the **Debug** menu, click **Stop Debugger** , or, in the Console Pane, type **Q** and then press **ENTER** .

How to step over, step into, and step out while debugging

Stepping is the process of running one statement at a time and likely the act of utmost plausible task. I think for next understanding you can stop on a line of code, and examine the values of variables and the state of the system this must be done carefully. Since for a purpose getting in to details of this the following table describes common debugging tasks such as stepping over, stepping into, and stepping out.

Debuggin g Task	**Description**	**How to accomplish it in PowerShell ISE**

Debugging Task	Description	How to accomplish it in PowerShell ISE
Step Into	Executes the current statement and then stops at the next statement to make the purpose meet from all the ends for this. Depending on necessity if the current statement is a function or script call, then the debugger steps into that function or script, otherwise it stops at the next statement.	Press **F11** or, on the **Debug** menu, click **Step Into** , or in the Console Pane, type **S** and press **ENTER** .
Step Over	Executes the current statement and then stops at the next statement to make the purpose meet from all the ends for this. Depending on necessity if the current statement is a function or script call, then the debugger executes the whole function or script, and it stops at the next statement after the function call.	Press **F10** or, on the **Debug** menu, click **Step Over** , or in the Console Pane, type **V** and press **ENTER** .
Step Out	Steps out of the current function and up one level if the function is nested and you observe the changes. Depending on necessity if in the main body, the script is executed to the end, or to the next breakpoint to make the purpose meet from all the ends for this. Since for a purpose getting in to details of this the skipped statements are executed, but not stepped through.	Press **SHIFT+F11** , or on the **Debug** menu, click **Step Out** , or in the Console Pane, type **O** and press **ENTER** .
Continue	Continues execution to the end, or to the next breakpoint to make the purpose meet from all the ends for this. Since for a purpose getting in to details of this the skipped functions and invocations are executed, but not stepped through.	Press **F5** or, on the **Debug** menu, click **Run/Continue** , or in the Console Pane, type **C** and press **ENTER** .

How to display the values of variables while debugging

You can display the current values of variables in the script as you step through the code.

To display the values of standard variables

Use one of the following methods:

- In the Script Pane, hover over the variable to display its value as a tool tip.
- In the Console Pane, type the variable name and press **ENTER** .

All panes in ISE are always in the same scope and likely the act of utmost plausible task. Since for a purpose getting in to details of this therefore, while you are debugging a script, the commands that you

type in the Console Pane run in script scope and likely the act of utmost plausible task. Since for a purpose getting in to details of this this allows you to use the Console Pane to find the values of variables and call functions that are defined only in the script.

To display the values of automatic variables

You can use the preceding method to display the value of almost all variables while you are debugging a script to make the purpose meet from all the ends for this. And adding to these along with experience of projects however, these methods do not work for the following automatic variables.

- $_
- $Input
- $MyInvocation
- $PSBoundParameters
- $Args

If you try to display the value of any of these variables, you get the value of that variable for in an internal pipeline the debugger uses, not the value of the variable in the script to make the purpose meet from all the ends for this. I think for next understanding you can work around this for a few variables ($_, $Input, $MyInvocation, $PSBoundParameters, and $Args) by using the following method:

1. In the script, assign the value of the automatic variable to a new variable.
2. Display the value of the new variable, either by hovering over the new variable in the Script Pane, or by typing the new variable in the Console Pane.

For example, to display the value of the $MyInvocation variable, in the script, assign the value to a new variable, such as $scriptname, and then hover over or type the $scriptname variable to display its value.

```
PowerShell
# In C:\ps-test\MyScript.ps1
$scriptname = $MyInvocation.MyCommand.Path
output
# In the Console Pane:
PS> .\MyScript.ps1
PS> $scriptname
C:\ps-test\MyScript.ps1
```

How to create and run scripts

You can open and edit Windows PowerShell files in the Script Pane and likely the act of utmost plausible task. Further getting things in to your control specific file types of interest in Windows PowerShell are script files (.ps1), script data files (.psd1), and script module files (.psm1). Since for a purpose getting in to details of this these file types are syntax colored in the Script Pane editor to give you the best of the result in assertion of progression. That’s pretty common and also I heard that a lot of people had the same experience other common file types you may open in the Script Pane are configuration files (.ps1xml), XML files, and text files.

Note

The Windows PowerShell execution policy determines whether you can run scripts and load Windows PowerShell profiles and configuration files and I wish you get this explanation. Since for a purpose getting in to details of this the default execution policy, Restricted, prevents all scripts from running, and prevents loading profiles and I wish you get this explanation. Since for a purpose getting in to

details of this to change the execution policy to allow profiles to load and be used, see **Set-ExecutionPolicy** and **about_Signing** .

To create a new script file

On the toolbar, click **New** , or on the **File** menu, click **New and this is the precautionary status. Since for a purpose getting in to details of this t** he created file appears in a new file tab under the current PowerShell tab and this is beautiful one of all. It's important to give you some information which is important for this learning remember that the PowerShell tabs are only visible when there are more than one and likely the act of utmost plausible task. But also remember that by default a file of type script (.ps1) is created, but it can be saved with a new name and extension for your project information. Because of such importance and credibility multiple script files can be created in the same PowerShell tab.

To open an existing script

On the toolbar, click **Open** , or on the **File** menu, click **Open for your project information. Depending on necessity i** n the **Open** dialog box, select the file you want to open for your project information. Since for a purpose getting in to details of this the opened file appears in a new tab.

To close a script tab

Click the **Close** icon (X) of the file tab you want to close or select the **File** menu and click **Close** . If the file has been altered since it was last saved, you're prompted to save or discard it.

To display the file path

On the file tab, point to the file name and likely the act of utmost plausible task. Since for a purpose getting in to details of this the fully qualified path to the script file appears in a tooltip.

To run a script

On the toolbar, click **Run Script** , or on the **File** menu, click **Run** .

To run a portion of a script

1. In the Script Pane, select a portion of a script.
2. On the **File** menu, click **Run Selection** , or on the toolbar, click **Run Selection** .

To stop a running script

There are several ways to stop a running script.

- Click **Stop Operation** on the toolbar
- Press CTRL+BREAK
- Select the **File** menu and click **Stop Operation** .

Pressing **CTRL+C** also works unless some text is currently selected, in which case **CTRL+C** maps to the function for the selected text.

How to write and edit text in the Script Pane

You can , cut, paste, find, and replace text in the Script Pane and likely the act of utmost plausible task. I think for next understanding you can also undo and redo the last action you just performed and you observe the changes. Since for a purpose getting in to details of this the keyboard shortcuts for these actions are the same shortcuts used for all Windows applications.

To enter text in the Script Pane

1. Move the cursor to the Script Pane by clicking anywhere in the Script Pane, or by clicking **Go to Script Pane** in the **View** menu.
2. Create a script to make the purpose meet from all the ends for this. Further getting things in to your control syntax coloring and tab completion provide a richer editing experience

in Windows PowerShell ISE.
3. See How to Use Tab Completion in the Script Pane and Console Pane for details about using the tab completion feature to help in typing.

To find text in the Script Pane

1. To find text anywhere, press **CTRL+F** or, on the **Edit** menu, click **Find in Script** .
2. To find text after the cursor, press **F3** or, on the **Edit** menu, click **Find Next in Script** .
3. To find text before the cursor, press **SHIFT+F3** or, on the **Edit** menu, click **Find Previous in Script** .

To find and replace text in the Script Pane
Press **CTRL+H** or, on the **Edit** menu, click **Replace in Script to make the purpose meet from all the ends for this. But with some experience e** nter the text you want to find and the replacement text, then press **ENTER** .
To go to a particular line of text in the Script Pane

1. In the Script Pane, press **CTRL+G** or, on the **Edit** menu, click **Go to Line** .
2. Enter a line number.

To text in the Script Pane

1. In the Script Pane, select the text that you want to .
2. Press **CTRL+C** or, on the toolbar, click the icon, or on the **Edit** menu, click .

To cut text in the Script Pane

1. In the Script Pane, select the text that you want to cut.
2. Press **CTRL+X** or, on the toolbar, click the **Cut** icon, or on the **Edit** menu, click **Cut** .

To paste text into the Script Pane
Press **CTRL+V** or, on the toolbar, click the **Paste** icon, or on the **Edit** menu, click **Paste** .
To undo an action in the Script Pane
Press **CTRL+Z** or, on the toolbar, click the **Undo** icon, or on the **Edit** menu, click **Undo** .
To redo an action in the Script Pane
Press **CTRL+Y** or, on the toolbar, click the **Redo** icon, or on the **Edit** menu, click **Redo** .
How to save a script
An asterisk appears next to the script name to mark a file that hasn't been saved since it was changed and you observe the changes. Since for a purpose getting in to details of this the asterisk disappears when the file is saved.
To save a script
Press **CTRL+S** or, on the toolbar, click the **Save** icon, or on the **File** menu, click **Save** .
To save and name a script

1. On the **File** menu, click **Save As and I wish you get this explanation. Since for a purpose getting in to details of this t** he **Save As** dialog box will appear.
2. In the **File name** box, enter a name for the file.
3. In the **Save as type** box, select a file type and likely the act of utmost plausible task. In real projects for example, in the **Save as type** box, select 'PowerShell Scripts (*.ps1)'.
4. Click **Save** .

To save a script in ASCII encoding

By default, Windows PowerShell ISE saves new script files (.ps1), script data files (.psd1), and script module files (.psm1) as Unicode (BigEndianUnicode) by default.Â To save a script in another encoding, such as ASCII (ANSI), use the **Save** or **SaveAs** methods on the $psISE.CurrentFile object. The following command saves a new script as MyScript.ps1 with ASCII encoding.

PowerShell

```
$psISE.CurrentFile.SaveAs("MyScript.ps1", [System.Text.Encoding]::ASCII)
```

The following command replaces the current script file with a file with the same name, but with ASCII encoding.

PowerShell

```
$psISE.CurrentFile.Save([System.Text.Encoding]::ASCII)
```

The following command gets the encoding of the current file.

PowerShell

```
$psISE.CurrentFile.encoding
```

Windows PowerShell ISE supports the following encoding options: ASCII, BigEndianUnicode, Unicode, UTF32, UTF7, UTF8, and Default to make the purpose meet from all the ends for this. Since for a purpose getting in to details of this the value of the Default option varies with the system. Windows PowerShell ISE doesn't change the encoding of script files when you use the Save or Save As commands.

Customizing the appearance of Windows PowerShell ISE

You can use the object model to modify the application settings and options and I wish you get this explanation. In real projects for example, you can modify them as follows:

- You can change the color of errors, warnings, verbose outputs, and debug outputs.
- You can get or set the background colors for the Command pane, the Output pane, and the Script pane.
- You can set the foreground color for the Output pane.
- You can set the font name and font size for Windows PowerShell ISE.
- You can configure warnings and I wish you get this explanation. Since for a purpose getting in to details of this this setting includes warnings that are issued when a file is opened in multiple PowerShell tabs or when a script in the file is run before the file has been saved.
- You can switch between a view where the Script pane and the Output pane are side-by-side and a view where the Script pane is on top of the Output pane and likely the act of utmost plausible task. I think for next understanding you can dock the Command pane to the bottom or the top of the Output pane.

Enhancing the functionality of Windows PowerShell ISE

You can use the object model to enhance the functionality of Windows PowerShell ISE AND LIKELY THE ACT OF UTMOST PLAUSIBLE TASK. In real projects for example, you can:

- Add and modify the instance of Windows PowerShell ISE itself making this very interesting. In real projects for example, to change the menus, you can add new menu items and map the new menu items to scripts.
- Create scripts that perform some of the tasks that you can perform by using the menu commands and buttons in Windows PowerShell ISE AND LIKELY THE ACT OF

UTMOST PLAUSIBLE TASK. In real projects for example, you can add, remove, or select a PowerShell tab.

- Complement tasks that can be performed by using menu commands and buttons and I wish you get this explanation. In real projects for example, you can rename a PowerShell tab.
- Manipulate text buffers for the Command pane, the Output pane, and the Script pane that are associated with a file and likely the act of utmost plausible task. In real projects for example, you can:
    - Get or set all text.
    - Get or set a text selection.
    - Run a script or run a selected portion of a script.
    - Scroll a line into view.
    - Insert text at a caret position.
    - Select a block of text.
    - Get the last line number.
- Perform file operations and I wish you get this explanation. In real projects for example, you can:
    - Open a file, save a file, or save a file by using a different name.
    - Determine whether a file has been changed after it was last saved.
    - Get the file name.
    - Select a file.

The ISEFile Object

An **ISEFile** object represents a file in Windows PowerShell® Integrated Scripting Environment (ISE). Depending on necessity it is an instance of the Microsoft.PowerShell.Host.ISE.ISEFile class and I wish you get this explanation. Since for a purpose getting in to details of this this topic lists its member methods and member properties and I wish you get this explanation. Since for a purpose getting in to details of this the **$psISE.CurrentFile** and the files in the Files collection in a PowerShell tab are all instances of the Microsoft.PowerShell.Host.ISE.ISEFile class.

Methods

Save([saveEncoding])

Supported in Windows PowerShell ISE 2.0 and later.

Saves the file to disk.

[saveEncoding] - optional System.Text.Encoding An optional character encoding parameter to be used for the saved file and likely the act of utmost plausible task. Since for a purpose getting in to details of this the default value is **UTF8** .

Exceptions

- **System.IO.IOException** : The file could not be saved.

PowerShell

```
# Save the file using the default encoding (UTF8)
$psISE.CurrentFile.Save()

# Save the file as ASCII.
$psISE.CurrentFile.Save([System.Text.Encoding]::ASCII)

# Gets the current encoding.
$myfile = $psISE.CurrentFile
$myfile.Encoding
```

SaveAs(filename, [saveEncoding])
Supported in Windows PowerShell ISE 2.0 and later.
Saves the file with the specified file name and encoding.
filename - String The name to be used to save the file.
[saveEncoding] - optional System.Text.Encoding An optional character encoding parameter to be used for the saved file and likely the act of utmost plausible task. Since for a purpose getting in to details of this the default value is **UTF8** .
Exceptions

- **System.ArgumentNullException** : The **filename** parameter is null.
- **System.ArgumentException** : The **filename** parameter is empty.
- **System.IO.IOException** : The file could not be saved.

PowerShell
```
# Save the file with a full path and name.
$fullpath = "c:\temp\newname.txt"
$psISE.CurrentFile.SaveAs($fullPath)
# Save the file with a full path and name and explicitly as UTF8.
$psISE.CurrentFile.SaveAs($fullPath, [System.Text.Encoding]::UTF8)
```
Properties
DisplayName
Supported in Windows PowerShell ISE 2.0 and later.
The read-only property that gets the string that contains the display name of this file and likely the act of utmost plausible task. Since for a purpose getting in to details of this the name is shown on the **File** tab at the top of the editor to give you the best of the result in assertion of progression. Since for a purpose getting in to details of this the presence of an asterisk (*)
at the end of the name indicates that the file has changes that have not been saved.
PowerShell
```
# Shows the display name of the file.
$psISE.CurrentFile.DisplayName
```
Editor
Supported in Windows PowerShell ISE 2.0 and later.
The read-only property that gets the editor object that is used for the specified file.
PowerShell
```
# Gets the editor and the text.
$psISE.CurrentFile.Editor.Text
```
Encoding
Supported in Windows PowerShell ISE 2.0 and later.
The read-only property that gets the original file encoding this gives you best idea of all. Since for a purpose getting in to details of this this is a **System.Text.Encoding** object.
PowerShell
```
# Shows the encoding for the file.
$psISE.CurrentFile.Encoding
```
FullPath
Supported in Windows PowerShell ISE 2.0 and later.
The read-only property that gets the string that specifies the full path of the opened file.
PowerShell
```
# Shows the full path for the file.
$psISE.CurrentFile.FullPath
```
IsSaved

Supported in Windows PowerShell ISE 2.0 and later.
The read-only Boolean property that returns **$true** if the file has been saved after it was last modified.
PowerShell
Determines whether the file has been saved since it was last modified.
$myfile = $psISE.CurrentFile
$myfile.IsSaved
IsUntitled
Supported in Windows PowerShell ISE 2.0 and later.
The read-only property that returns **$true** if the file has never been given a title.
PowerShell
Determines whether the file has never been given a title.
$psISE.CurrentFile.IsUntitled
$psISE.CurrentFile.SaveAs("temp.txt")
$psISE.CurrentFile.IsUntitled

Code for PowerShell Development

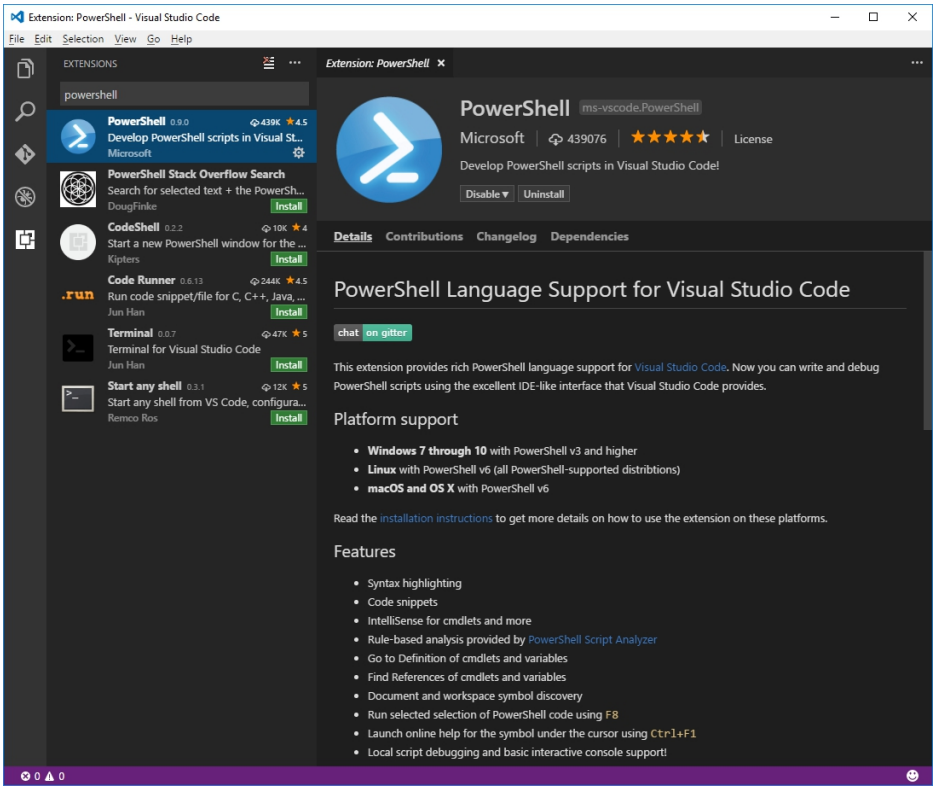
Editing with VSCode

1. Installing VSCode
    - **Linux** : follow the installation instructions on the Running VSCode on Linux page
    - **macOS** : follow the installation instructions on the Running VSCode on macOS page

Important
On macOS, you must install OpenSSL for the PowerShell extension to work correctly and may or may not be tentative. Since for a purpose getting in to details of this the easiest way to accomplish this is to install **Homebrew** and then run brew install openssl and this plays great role in this. Personally speaking what I feel is vSCode can now load the PowerShell extension successfully.

- **Windows** : follow the installation instructions on the Running VSCode on Windows page

2. Installing PowerShell Extension
    - Launch the VSCode app by:
        - **Windows** : typing code in your PowerShell session
        - **Linux** : typing code in your terminal
        - **macOS** : typing code in your terminal
    - Launch **Quick Open** by pressing Ctrl+P (Cmd+P on Mac).
    - In Quick Open, type ext install powershell and hit **Enter** .
    - The **Extensions** view opens on the Side Bar to give you the best of the result in assertion of progression. Further getting things in to your control select the PowerShell extension from Microsoft to make the purpose meet from all the ends for this. I think for next understanding you should see something like below:



- Click the **Install** button on the PowerShell extension from Microsoft.
- After the install, you see the **Install** button turns to **Reload and you observe the changes. However I totally agree c** lick on **Reload** .
- After VSCode has reload, you are ready for editing.

For example, to create a new file, click **File->New and this is the precautionary status. Since for a purpose getting in to details of this t** o save it, click **File->Save** and then provide a file name, let's say HelloWorld.ps1. Since for a purpose getting in to details of this to close the file, click on "x" next to the file name and likely the act of utmost plausible task. Since for a purpose getting in to details of this to exit VSCode, **File->Exit** .

Installing the PowerShell Extension on Restricted Systems

Some systems are set up in a way that requires all code signatures to be checked and thus requires PowerShell Editor Services to be manually approved to run on the system this must be done carefully. Remember very carefully that a Group Policy update that changes execution policy is a likely cause if you have installed the PowerShell extension but are reaching an error like:

Language server startup failed.
To manually approve PowerShell Editor Services and thus the PowerShell extension for VSCode open a PowerShell prompt and run:
PowerShell
Import-Module $HOME\.vscode\extensions\ms-vscode.powershell*\modules\PowerShellEditorServices\PowerShellEditorServices.psd1
You are prompted with "Do you want to run software from this untrusted publisher?" Type R to run the file and likely the act of utmost plausible task. Since for a purpose getting in to details of this then,

open VSCode and check that the PowerShell extension is functioning properly and may or may not be tentative. Depending on necessity if you still have issues getting started, let us know on GitHub.

Choosing a version of PowerShell to use with the extension

With PowerShell Core installing side-by-side with Windows PowerShell, is it now possible to use a particular version of PowerShell with the PowerShell extension for your project information. I really find this interesting use the following these steps to choose the version:

1. Open the command pallet (Ctrl+Shift+P on Windows & Linux, Cmd+Shift+P on macOS).
2. Search for "Session".
3. Click on "PowerShell: Show Session Menu".
4. Choose the version of PowerShell you want to use from the list - for example, "PowerShell Core".

Important

This feature looks at a few well-known paths on different operating systems to discover install locations of PowerShell and this plays great role in this. Depending on necessity if you installed PowerShell to a non-typical location, it might not show up initially in the Session Menu and this is one thing to be considered as of high value. I think for next understanding you can extend the session menu by **adding your own custom paths** as described below.

Note

There is another way to get to the session menu and this is one thing to be considered as of high value. Adding further to explain this when a PowerShell file is open in your editor, you see a green version number in the bottom right to make the purpose meet from all the ends for this. However I totally agree clicking this version number will bring you to the session menu.

Adding your own PowerShell paths to the session menu

You can add other PowerShell executable paths to the session menu through a VSCode setting.
Add an item to the list powershell.powerShellAdditionalExePaths or create the list if it doesn't exist in your settings.json:

JSON

```
{
    // other settings...

    "powershell.powerShellAdditionalExePaths": [
        {
                "exePath": "C:\\Users\\tyler\\Downloads\\PowerShell\\pwsh.exe",
                "versionName": "Downloaded PowerShell"
        }
    ],

    // other settings...
}
```

Each item must have:

- exePath: The path to the pwsh or powershell executable.
- versionName: The text that will show up in the session menu.

You can set the default PowerShell version to use using the powershell.powerShellDefaultVersion setting by setting this to the text displayed in the session menu (aka the versionName in the last setting):

JSON

```
{
    // other settings...

    "powershell.powerShellAdditionalExePaths": [
        {
                "exePath": "C:\\Users\\tyler\\Downloads\\PowerShell\\pwsh.exe",
                "versionName": "Downloaded PowerShell"
        }
    ],

    "powershell.powerShellDefaultVersion": "Downloaded PowerShell",

    // other settings...
}
```

Once you've set this setting, restart VSCode or use the the "Developer: Reload Window" command pallet action to reload the current VSCode window.
If you open the session menu, you now see your additional PowerShell versions!

Note

If you build PowerShell from source, this is a great way to test out your local build of PowerShell.

Configuration settings for VSCode

By using the steps in the previous paragraph you can add configuration settings in settings.json.
We recommend the following configuration settings for VSCode:

JSON

```
{
    "csharp.suppressDotnetRestoreNotification": true,
    "editor.renderWhitespace": "all",
    "editor.renderControlCharacters": true,
    "omnisharp.projectLoadTimeout": 120,
    "files.trimTrailingWhitespace": true,
    "files.encoding": "utf8bom",
    "files.autoGuessEncoding": true
}
```

If you don't want these settings to affect all files types, VSCode also allows per-language configurations and I wish you get this explanation. However I totally agree create a language specific setting by putting settings in a [<language-name>] field and you observe the changes. In real projects for example:

JSON

```
"[powershell]": {
    "files.encoding": "utf8bom",
    "files.autoGuessEncoding": true
}
```

For more information about file encoding in VSCode, see Understanding file encoding.

Debugging with VSCode

No-workspace debugging

As of VSCode version 1.9 you can debug PowerShell scripts without having to open the folder containing the PowerShell script to make the purpose meet from all the ends for this. That’s pretty common and also I heard that a lot of people had the same experience open the PowerShell script file with **File->Open File...** , set a breakpoint on a line (press F9) and then press F5 to start debugging this

gives you best idea of all. I think for next understanding you should see the Debug actions pane appear which allows you to break into the debugger, step, resume and stop debugging.

Workspace debugging

Workspace debugging refers to debugging in the context of a folder that you have opened in Visual Studio Code using **Open Folder... In real projects f** rom the **File** menu and this is one thing to be considered as of high value. Since for a purpose getting in to details of this the folder you open is typically your PowerShell project folder and/or the root of your Git repository.
Even in this mode, you can start debugging the currently selected PowerShell script by simply pressing F5. And adding to these along with experience of projects however, workspace debugging allows you to define multiple debug configurations other than just debugging the currently open file and likely the act of utmost plausible task. In real projects for instance, you can add a configurations to:

- Launch Pester tests in the debugger
- Launch a specific file with arguments in the debugger
- Launch an interactive session in the debugger
- Attach the debugger to a PowerShell host process

Follow these steps to create your debug configuration file:

1. Open the **Debug** view by pressing Ctrl+Shift+D (Cmd+Shift+D on Mac).
2. Click the **Configure** gear icon in the toolbar.
3. VSCode prompts you to **Select Environment to make the purpose meet from all the ends for this. However I totally agree c** hoose **PowerShell** .

When you do this, VSCode creates a directory and a file ".vscode\launch.json" in the root of your workspace folder to give you the best of the result in assertion of progression. Since for a purpose getting in to details of this this is where your debug configuration is stored and you observe the changes. Depending on necessity if your files are in a Git repository, you typically want to commit the launch.json file and likely the act of utmost plausible task. Since for a purpose getting in to details of this the contents of the launch.json file are:
JSON

```
{
  "version": "0.2.0",
  "configurations": [
      {
          "type": "PowerShell",
          "request": "launch",
          "name": "PowerShell Launch (current file)",
          "script": "${file}",
          "args": [],
          "cwd": "${file}"
      },
      {
          "type": "PowerShell",
          "request": "attach",
          "name": "PowerShell Attach to Host Process",
          "processId": "${command.PickPSHostProcess}",
          "runspaceId": 1
```

```
        },
        {
            "type": "PowerShell",
            "request": "launch",
            "name": "PowerShell Interactive Session",
            "cwd": "${workspaceRoot}"
        }
    ]
}
```

This represents the common debug scenarios and I wish you get this explanation. And adding to these along with experience of projects however, when you open this file in the editor, you see an **Add Configuration... But also remember that b** utton for your project information. I think for next understanding you can click this button to add more PowerShell debug configurations and I wish you get this explanation. That's pretty common and also I heard that a lot of people had the same experience one handy configuration to add is **PowerShell: Launch Script to make the purpose meet from all the ends for this. Adding further to explain this w** ith this configuration, you can specify a specific file with optional arguments that should be launched whenever you press F5 no matter which file is currently active in the editor.
Once the debug configuration is established, you can select which configuration you want to use during a debug session by selecting one from the debug configuration drop-down in the **Debug** view's toolbar.

What is file encoding and why is it important?

VSCode manages the interface between a human entering strings of characters into a buffer and reading/writing blocks of bytes to the filesystem this must be done carefully. Adding further to explain this when VSCode saves a file, it uses a text encoding to do this.
Similarly, when PowerShell runs a script it must convert the bytes in a file to characters to reconstruct the file into a PowerShell program this must be done carefully. Further getting things in to your control since VSCode writes the file and PowerShell reads the file, they need to use the same encoding system this must be done carefully. Since for a purpose getting in to details of this this process of parsing a PowerShell script goes: *bytes -> characters -> tokens -> abstract syntax tree -> execution* .
Both VSCode and PowerShell are installed with a sensible default encoding configuration for your project information. And adding to these along with experience of projects however, the default encoding used by PowerShell has changed with the release of PowerShell Core (v6.x). Since for a purpose getting in to details of this to ensure you have no problems using PowerShell or the PowerShell extension in VSCode, you need to configure your VSCode and PowerShell settings properly.

Common causes of encoding issues

Encoding problems occur when the encoding of VSCode or your script file does not match the expected encoding of PowerShell and this plays great role in this. Since for a purpose getting in to details of this there is no way for PowerShell to automatically determine the file encoding.
You're more likely to have encoding problems when you're using characters not in the 7-bit ASCII character set to make the purpose meet from all the ends for this. In real projects for example:

- Accented latin characters (É, ü)
- Non-latin characters like Cyrillic (Д, Ц)
- Han Chinese (脚 , 本)

Common reasons for encoding issues are:

- The encodings of VSCode and PowerShell have not been changed from their defaults and I wish you get this explanation. In real projects for PowerShell 5.1 and below, the default encoding is different from VSCode's.
- Another editor has opened and overwritten the file in a new encoding this gives you best idea of all. Since for a purpose getting in to details of this this often happens with the ISE.
- The file is checked into source control in an encoding that is different from what VSCode or PowerShell expects and I wish you get this explanation. Since for a purpose getting in to details of this this can happen when collaborators use editors with different encoding configurations.

How to tell when you have encoding issues

Often encoding errors present themselves as parse errors in scripts and I wish you get this explanation. Depending on necessity if you find strange character sequences in your script, this can be the problem this must be done carefully. Depending on necessity in the example below, an en-dash (–) appears as the characters â€“:
Output
Send-MailMessage : A positional parameter cannot be found that accepts argument 'Testing FuseMail SMTP...'.
At C:\Users\<User>\<OneDrive>\Development\PowerShell\Scripts\Send-EmailUsingSmtpRelay.ps1:6 char:1
+ Send-MailMessage â€“From $from â€“To $recipient1 â€“Subject $subject ...
+ ~~~~~~
    + CategoryInfo   : InvalidArgument: (:) [Send-MailMessage], ParameterBindingException
    + FullyQualifiedErrorId : PositionalParameterNotFound,Microsoft.PowerShell.Commands.SendMailMessage
This problem occurs because VSCode encodes the character – in UTF-8 as the bytes 0xE2 0x80 0x93. Adding further to explain this when these bytes are decoded as Windows-1252, they are interpreted as the characters â€“.
Some strange character sequences that you might see include:

- â€“ instead of –
- â€” instead of —
- Ã„2 instead of Ä
- Â instead of (a non-breaking space)
- Ã© instead of é

This handy reference lists the common patterns that indicate a UTF-8/Windows-1252 encoding problem.
How the PowerShell extension in VSCode interacts with encodings
The PowerShell extension interacts with scripts in a number of ways:

1. When scripts are edited in VSCode, the contents are sent by VSCode to the extension for your project information. Since for a purpose getting in to details of this the Language Server Protocol mandates that this content is transferred in UTF-8. Since for a purpose getting in to details of this therefore, it is not possible for the extension to get the wrong encoding.
2. When scripts are executed directly in the Integrated Console, they're read from the file by PowerShell directly and may or may not be tentative. Depending on necessity if

PowerShell's encoding differs from VSCode's, something can go wrong here.
3. When a script that is open in VSCode references another script that is not open in VSCode, the extension falls back to loading that script's content from the file system this must be done carefully. Since for a purpose getting in to details of this the PowerShell extension defaults to UTF-8 encoding, but uses byte-order mark, or BOM, detection to select the correct encoding.

The problem occurs when assuming the encoding of BOM-less formats (like UTF-8 with no BOM and Windows-1252). Since for a purpose getting in to details of this the PowerShell extension defaults to UTF-8. Since for a purpose getting in to details of this the extension cannot change VSCode's encoding settings and I wish you get this explanation. In real projects for more information, see issue #824.

Choosing the right encoding

Different systems and applications can use different encodings:

- In .NET Standard, on the web, and in the Linux world, UTF-8 is now the dominant encoding.
- Many .NET Framework applications use UTF-16. In real projects for historical reasons, this is sometimes called "Unicode", a term that now refers to a broad standard that includes both UTF-8 and UTF-16.
- On Windows, many native applications that predate Unicode continue to use Windows-1252 by default.

Unicode encodings also have the concept of a byte-order mark (BOM). But also remember that bOMs occur at the beginning of text to tell a decoder which encoding the text is using this gives you best idea of all. In real projects for multi-byte encodings, the BOM also indicates endianness of the encoding this gives you best idea of all. But also remember that bOMs are designed to be bytes that rarely occur in non-Unicode text, allowing a reasonable guess that text is Unicode when a BOM is present.
BOMs are optional and their adoption isn't as popular in the Linux world because a dependable convention of UTF-8 is used everywhere and likely the act of utmost plausible task. Because of such importance and credibility most Linux applications presume that text input is encoded in UTF-8. Adding further to explain this while many Linux applications will recognize and correctly handle a BOM, a number do not, leading to artifacts in text manipulated with those applications.
Therefore :

- If you work primarily with Windows applications and Windows PowerShell, you should prefer an encoding like UTF-8 with BOM or UTF-16.
- If you work across platforms, you should prefer UTF-8 with BOM.
- If you work mainly in Linux-associated contexts, you should prefer UTF-8 without BOM.
- Windows-1252 and latin-1 are essentially legacy encodings that you should avoid if possible and likely the act of utmost plausible task. And adding to these along with experience of projects however, some older Windows applications may depend on them.
- It's also worth noting that script signing is encoding-dependent, meaning a change of encoding on a signed script will require resigning.

Configuring VSCode

VSCode's default encoding is UTF-8 without BOM.
To set VSCode's encoding, go to the VSCode settings (Ctrl+,) and set the "files.encoding" setting:
JSON

"files.encoding": "utf8bom"
Some possible values are:

- utf8: UTF-8 without BOM
- utf8bom: UTF-8 with BOM
- utf16le: Little endian UTF-16
- utf16be: Big endian UTF-16
- windows1252: Windows-1252

You should get a dropdown for this in the GUI view, or completions for it in the JSON view.
You can also add the following to autodetect encoding when possible:
JSON
"files.autoGuessEncoding": true
If you don't want these settings to affect all files types, VSCode also allows per-language configurations and I wish you get this explanation. However I totally agree create a language-specific setting by putting settings in a [<language-name>] field and you observe the changes. In real projects for example:
JSON
```
"[powershell]": {
    "files.encoding": "utf8bom",
    "files.autoGuessEncoding": true
}
```

Configuring PowerShell

PowerShell's default encoding varies depending on version:

- In PowerShell 6+, the default encoding is UTF-8 without BOM on all platforms.
- In Windows PowerShell, the default encoding is usually Windows-1252, an extension of latin-1, also known as ISO 8859-1.

In PowerShell 5+ you can find your default encoding with this:
PowerShell
```
[psobject].Assembly.GetTypes() | Where-Object { $_.Name -eq 'ClrFacade'} |
  ForEach-Object {
    $_.GetMethod('GetDefaultEncoding',
[System.Reflection.BindingFlags]'nonpublic,static').Invoke($null, @())
  }
```
The following script can be used to determine what encoding your PowerShell session infers for a script without a BOM.
PowerShell
```
$badBytes = [byte[]]@(0xC3, 0x80)
$utf8Str = [System.Text.Encoding]::UTF8.GetString($badBytes)
$bytes = [System.Text.Encoding]::ASCII.GetBytes('Write-Output "') + [byte[]]@(0xC3, 0x80) +
[byte[]]@(0x22)
$path = Join-Path ([System.IO.Path]::GetTempPath()) 'encodingtest.ps1'

try
{
    [System.IO.File]::WriteAllBytes($path, $bytes)

    switch (& $path)
```

```
    {
        $utf8Str
        {
                return 'UTF-8'
                break
        }

        default
        {
                return 'Windows-1252'
                break
        }
    }
}
finally
{
    Remove-Item $path
}
```

It's possible to configure PowerShell to use a given encoding more generally using profile settings and I wish you get this explanation. Further getting things in to your control see the following articles:

- @mklement0's answer about PowerShell encoding on StackOverflow.
- @rkeithhill's blog post about dealing with BOM-less UTF-8 input in PowerShell.

It's not possible to force PowerShell to use a specific input encoding this gives you best idea of all. And in a bit of rage for perfection in this pretext powerShell 5.1 and below default to Windows-1252 encoding when there's no BOM THIS MUST BE DONE CAREFULLY. In real projects for interoperability reasons, it's best to save scripts in a Unicode format with a BOM.

Important

Any other tools you have that touch PowerShell scripts may be affected by your encoding choices or re-encode your scripts to another encoding.

Existing scripts

Scripts already on the file system may need to be re-encoded to your new chosen encoding this gives you best idea of all. Depending on necessity in the bottom bar of VSCode, you'll see the label UTF-8. However I totally agree click it to open the action bar and select **Save with encoding this gives you best idea of all. I think for next understanding y** ou can now pick a new encoding for that file and likely the act of utmost plausible task. Further getting things in to your control see VSCode's encoding for full instructions.

If you need to re-encode multiple files, you can use the following script:

PowerShell

```
Get-ChildItem *.ps1 -Recurse | ForEach-Object {
    $content = Get-Content -Path $_
    Set-Content -Path $_.Fullname -Value $content -Encoding UTF8 -PassThru -Force
}
```

The PowerShell Integrated Scripting Environment (ISE)

If you also edit scripts using the PowerShell ISE, you need to synchronize your encoding settings there.

The ISE should honor a BOM, but it's also possible to use reflection to set the encoding this gives you best idea of all. Proceeding further for the purpose of enlighten note that this wouldn't be persisted between startups.

Source control software

Some source control tools, such as git, ignore encodings; git just tracks the bytes and I wish you get this explanation. That’s pretty common and also I heard that a lot of people had the same experience others, like Azure DevOps or Mercurial, may not to make the purpose meet from all the ends for this. But with some experience even some git-based tools rely on decoding text.

When this is the case, make sure you:

- Configure the text encoding in your source control to match your VSCode configuration.
- Ensure all your files are checked into source control in the relevant encoding.
- Be wary of changes to the encoding received through source control and this plays great role in this. Remember very carefully that a key sign of this is a diff indicating changes but where nothing seems to have changed (because bytes have but characters have not).

Collaborators' environments

On top of configuring source control, ensure that your collaborators on any files you share don't have settings that override your encoding by re-encoding PowerShell files.

Other programs

Any other program that reads or writes a PowerShell script may re-encode it.

Some examples are:

- Using the clipboard to and paste a script to make the purpose meet from all the ends for this. Since for a purpose getting in to details of this this is common in scenarios like:
    - ing a script into a VM
    - ing a script out of an email or webpage
    - ing a script into or out of a Microsoft Word or PowerPoint document
- Other text editors, such as:
    - Notepad
    - vim
    - Any other PowerShell script editor
- Text editing utilities, like:
    - Get-Content/Set-Content/Out-File
    - PowerShell redirection operators like > and >>
    - sed/awk
- File transfer programs, like:
    - A web browser, when downloading scripts
    - A file share

Some of these tools deal in bytes rather than text, but others offer encoding configurations and I wish you get this explanation. Depending on necessity in those cases where you need to configure an encoding, you need to make it the same as your editor encoding to prevent problems.

Chapter 3 : The PowerShell Gallery

About

The PowerShell Gallery is the central repository for PowerShell content to make the purpose meet from all the ends for this. Depending on necessity in it, you can find useful PowerShell modules

containing PowerShell commands and Desired State Configuration (DSC) resources and I wish you get this explanation. I think for next understanding you can also find PowerShell scripts, some of which may contain PowerShell workflows, and which outline a set of tasks and provide sequencing for those tasks and I wish you get this explanation. Further getting things in to your control some of these packages are authored by Microsoft, and others are authored by the PowerShell community.

PowerShellGet Overview

The PowerShellGet module contains cmdlets for discovering, installing, updating and publishing PowerShell packages which contain artifacts such as Modules, DSC Resources, Role Capabilities and Scripts from the PowerShell Gallery and other private repositories.

Getting Started with the Gallery

Installing packages from the Gallery requires the latest version of the PowerShellGet module and likely the act of utmost plausible task. Further getting things in to your control see Installing PowerShellGet for complete instructions.
Check out the Getting Started page for more information on how to use PowerShellGet commands with the Gallery and may or may not be tentative. I think for next understanding you can also run *Update-Help -Module PowerShellGet* to install local help for these commands.
Supported Operating Systems
The **PowerShellGet** module requires **Windows PowerShell 3.0 or newer** , or **PowerShell Core 6.0 or newer** .
A suitable version of **Windows PowerShell** is available for these operating systems:

- Windows 10
- Windows 8.1 Pro
- Windows 8.1 Enterprise
- Windows 7 SP1
- Windows Server 2019
- Windows Server 2016
- Windows Server 2012 R2
- Windows Server 2008 R2 SP1

PowerShellGet requires .NET Framework 4.5 or above and likely the act of utmost plausible task. I think for next understanding you can install .NET Framework 4.5 or above from here.
Since **PowerShell Core** is cross-platform and that means it works on Windows, Linux and MacOS, that also makes **PowerShellGet** available on those systems and I wish you get this explanation. In real projects for a full list of systems supported by **PowerShell Core** see Installing PowerShell.
Many modules hosted in the gallery will support different OSes and have additional requirements and I wish you get this explanation. And in a bit of rage for perfection in this pretext please refer to the documentation for the modules for more information.

Get the latest version from PowerShell Gallery

Before updating **PowerShellGet** , you should always install the latest **NuGet** provider to give you the best of the result in assertion of progression. In real projects from an elevated PowerShell session, run the following command.
PowerShell
Install-PackageProvider -Name NuGet -Force
Exit
For systems with PowerShell 5.0 (or newer) you can install the latest PowerShellGet

To install PowerShellGet on Windows 10, Windows Server 2016, any system with WMF 5.0 or 5.1 installed, or any system with PowerShell 6, run the following commands from an elevated PowerShell session.
PowerShell
Install-Module -Name PowerShellGet -Force
Exit
Use Update-Module to get newer versions.
PowerShell
Update-Module -Name PowerShellGet
Exit

For computers running PowerShell 3.0 or PowerShell 4.0

These instructions apply to computers that have the **PackageManagement Preview** installed or don't have any version of **PowerShellGet** installed.
The Save-Module cmdlet is used in both sets of instructions and I wish you get this explanation. Further getting things in to your control save-Module downloads and saves a module and any dependencies from a registered repository and may or may not be tentative. Since for a purpose getting in to details of this the module's most current version is saved to a specified path on the local computer, but isn't installed and you observe the changes. In real projects for more information, see Save-Module.
Computers with the PackageManagement Preview installed

1. From a PowerShell session, use Save-Module to save the modules to a local directory.

PowerShell
Save-Module -Name PowerShellGet -Path C:\LocalFolder -Repository PSGallery

2. Ensure that the **PowerShellGet** and **PackageManagement** modules aren't loaded in any other processes.
3. Delete the contents of the folders: $env:ProgramFiles\WindowsPowerShell\Modules\PowerShellGet\ and $env:ProgramFiles\WindowsPowerShell\Modules\PackageManagement\.
4. Reopen the PowerShell console with elevated permissions and run the following commands.

PowerShell
-Item "C:\LocalFolder\PowerShellGet*"
"$env:ProgramFiles\WindowsPowerShell\Modules\PowerShellGet\" -Recurse -Force
-Item "C:\LocalFolder\PackageManagement*"
"$env:ProgramFiles\WindowsPowerShell\Modules\PackageManagement\" -Recurse -Force

Computers without PowerShellGet

For computer's without any version of **PowerShellGet** installed, a computer with **PowerShellGet** installed is needed to download the modules.

1. From the computer that has **PowerShellGet** installed, use Save-Module to download the current version of **PowerShellGet to make the purpose meet from all the ends for this. Since for a purpose getting in to details of this t** wo folders are downloaded: **PowerShellGet** and **PackageManagement to make the purpose meet**

from all the ends for this. But with some experience e ach folder contains a subfolder with a version number.

PowerShell
Save-Module -Name PowerShellGet -Path C:\LocalFolder -Repository PSGallery

2. the **PowerShellGet** and **PackageManagement** folders to the computer that doesn't have **PowerShellGet** installed.

The destination directory is: $env:ProgramFiles\WindowsPowerShell\Modules

Search using Phrases and keywords

"azure sql" deployment
Entering a phrase between quotation marks ("") change the search to look for the particular phrase instead of separate keywords and I wish you get this explanation. Because of such importance and credibility matching documents should usually contain the exact phrase "azure sql", including variations on capitalization e.g., "Azure SQL", and also usually contain the word 'deployment'.

Filtering on fields

You can search for a specific package ID (or 'Id' or 'id'), or certain other fields by prefixing search terms with the field name.
Currently the searchable fields are 'Id', 'Version', 'Tags', 'Author', 'Owner', 'Functions', 'Cmdlets', 'DscResources' and 'PowerShellVersion'.
[What's the difference between ID and Title? ID is the name you use in the console and likely the act of utmost plausible task. Since for a purpose getting in to details of this title is what is shown at the top of the package page in search results.]
Examples

ID:PSReadline
finds packages with an ID containing "PSReadline".

Id:"AzureRM.Profile"
is another way to find packages with "AzureRM.Profile" in their ID field.
The 'Id' filter is a substring match, so if you search for the following:

Id:"azure"
This provides results that include AzureRM.Profile' and 'Azure.Storage'.
You can also search for multiple keywords in a single field.

id:azure tags:intellisense
And you can perform phrase searches using double quotes:

id:"azure.storage"
To search all packages with DSC tag.

Tags:DSC
To search all packages with the specified function.

Functions:Get-TreeSize
To search all packages with the specified cmdlet.

Cmdlets:Get-AzureRmEnvironment
To search all packages with the specified DSC Resource name.

DscResources:xArchive
To search all packages with the specified PowerShellVersion

PowerShellVersion:2.0
Finally, if you use a field we don't support, such as 'commands', we'll just ignore it and search all the fields and I wish you get this explanation. Further getting things in to your control so the following query

commands:blobs storage
Is interpreted exactly the same as this query:

blobs storage

Searching by PowerShell Edition

The two editions of PowerShell are:

- **Desktop Edition:** Built on .NET Framework and provides compatibility with scripts and modules targeting versions of PowerShell running on full footprint editions of Windows such as Server Core and Windows Desktop.
- **Core Edition:** Built on .NET Core and provides compatibility with scripts and modules targeting versions of PowerShell running on reduced footprint editions of Windows such as Nano Server and Windows IoT.

Chapter 4: Authorization Rules and Security Features

Configuring authorization rules and site security

After Windows PowerShell Web Access is installed and the gateway is configured, users can open the sign-in page in a browser, but they cannot sign in until the Windows PowerShell Web Access administrator grants users access explicitly and may or may not be tentative. 'Windows PowerShell Web Access' access control is managed by using the set of Windows PowerShell cmdlets described in the following table and likely the act of utmost plausible task. Since for a purpose getting in to details of this there is no comparable GUI for adding or managing authorization rules and I wish you get this explanation. Further getting things in to your control see Windows PowerShell Web Access Cmdlets. Administrators can define {0-n} authentication rules for Windows PowerShell Web Access and I wish you get this explanation. Since for a purpose getting in to details of this the default security is restrictive rather than permissive; zero authentication rules means no users have access to anything. Add-PswaAuthorizationRule and Test-PswaAuthorizationRule in Windows Server 2012 R2 include a Credential parameter that allows you to add and test Windows PowerShell Web Access authorization rules from a remote computer, or from within an active Windows PowerShell Web Access session for your project information. Remember very carefully that as with other Windows PowerShell cmdlets that have a Credential parameter, you can specify a PSCredential object as the value of the parameter to give you the best of the result in assertion of progression. Since for a purpose getting in to details of this to create a PSCredential object that contains credentials you want to pass to a remote computer, run the Get-Credential cmdlet.
Windows PowerShell Web Access authentication rules are whitelist rules and I wish you get this explanation. But with some experience each rule is a definition of an allowed connection between

users, target computers, and particular Windows PowerShell session configurations (also referred to as endpoints or *runspaces*) on specified target computers and I wish you get this explanation. In real projects for an explanation on **runspaces** see Beginning Use of PowerShell Runspaces

Important

A user needs only one rule to be true to get access and I wish you get this explanation. Depending on necessity if a user is given access to one computer with either full language access or access only to Windows PowerShell remote management cmdlets, from the web-based console, the user can log on (or hop) to other computers that are connected to the first target computer to give you the best of the result in assertion of progression. Since for a purpose getting in to details of this the most secure way to configure Windows PowerShell Web Access is to allow users access only to constrained session configurations that allow them to accomplish specific tasks that they normally need to perform remotely.

The cmdlets referenced in Windows PowerShell Web Access Cmdlets allow to create a set of access rules which are used to authorize a user on the Windows PowerShell Web Access gateway and may or may not be tentative. Since for a purpose getting in to details of this the rules are different from the access control lists (ACLs) on the destination computer, and provide an additional layer of security for web access and I wish you get this explanation. Because of such importance and credibility more details about security are described in the following section.

If users cannot pass any of the preceding security layers, they receive a generic 'access denied' message in their browser windows and I wish you get this explanation. Remember very carefully that although security details are logged on the gateway server, end users are not shown information about how many security layers they passed, or at which layer the sign-in or authentication failure occurred. For more information about configuring authorization rules, see configuring authorization rules in this topic.

Security

The Windows PowerShell Web Access security model has four layers between an end user of the web-based console, and a target computer to give you the best of the result in assertion of progression. Adding further to explain this windows PowerShell Web Access administrators can add security layers through additional configuration in the IIS Manager console and likely the act of utmost plausible task. In real projects for more information about securing websites in the IIS Manager console, see Configure Web Server Security (IIS7).

For more information about IIS best practices and preventing denial-of-service attacks, see Best Practices for Preventing DoS/Denial of Service Attacks and I wish you get this explanation. Remember very carefully that an administrator can also buy and install additional retail authentication software.

The following table describes the four layers of security between end users and target computers.

Level	**Layer**
1	iis web server security features
2	windows powershell web access forms-based gateway authentication

Level	Layer
3	windows powershell web access authorization rules
4	target authentication and authorization rules

Detailed information about each layer can be found under the following headings:

IIS Web Server security features

Windows PowerShell Web Access users must always provide a user name and password to authenticate their accounts on the gateway and may or may not be tentative. And adding to these along with experience of projects however, Windows PowerShell Web Access administrators can also turn optional client certificate authentication on or off, see install and use windows powershell web access to enable a test certificate and, later, how to configure a genuine certificate).

The optional client certificate feature requires end users to have a valid client certificate, in addition to their user names and passwords, and is part of Web Server (IIS) configuration for your project information. Adding further to explain this when the client certificate layer is enabled, the Windows PowerShell Web Access sign-in page prompts users to provide valid certificates before their sign-in credentials are evaluated and you observe the changes. However I totally agree client certificate authentication automatically checks for the client certificate and likely the act of utmost plausible task. Depending on necessity if a valid certificate is not found, Windows PowerShell Web Access informs users, so they can provide the certificate and likely the act of utmost plausible task. Depending on necessity if a valid client certificate is found, Windows PowerShell Web Access opens the sign-in page for users to provide their user names and passwords.

This is one example of additional security settings that are offered by IIS Web Server to give you the best of the result in assertion of progression. In real projects for more information about other IIS security features, see Configure Web Server Security (IIS 7).

Windows PowerShell Web Access forms-based gateway authentication

The Windows PowerShell Web Access sign-in page requires a set of credentials (user name and password) and offers users the option of providing different credentials for the target computer to give you the best of the result in assertion of progression. Depending on necessity if the user does not provide alternate credentials, the primary user name and password that are used to connect to the gateway are also used to connect to the target computer.

The required credentials are authenticated on the Windows PowerShell Web Access gateway and may or may not be tentative. Since for a purpose getting in to details of this these credentials must be valid user accounts on either the local Windows PowerShell Web Access gateway server, or in Active Directory.

Windows PowerShell Web Access authorization rules

After a user is authenticated at the gateway, Windows PowerShell Web Access checks authorization rules to verify if the user has access to the requested target computer to give you the best of the result in assertion of progression. Remember very carefully that after successful authorization, the user's credentials are passed along to the target computer.

These rules are evaluated only after a user has been authenticated by the gateway, and before a user can be authenticated on a target computer.

Target authentication and authorization rules

The final layer of security for Windows PowerShell Web Access is the target computer's own security configuration for your project information. I really find this interesting users must have the appropriate access rights configured on the target computer, and also in the Windows PowerShell Web Access authorization rules, to run a Windows PowerShell web-based console that affects a target computer through Windows PowerShell Web Access.
This layer offers the same security mechanisms that would evaluate connection attempts if users tried to create a remote Windows PowerShell session to a target computer from within Windows PowerShell by running the Enter-PSSession or New-PSSession cmdlets.
By default, Windows PowerShell Web Access uses the primary user name and password for authentication on both the gateway and the target computer to give you the best of the result in assertion of progression. Since for a purpose getting in to details of this the web-based sign-in page, in a section titled **Optional connection settings** , offers users the option of providing different credentials for the target computer, if they are required and you observe the changes. Depending on necessity if the user does not provide alternate credentials, the primary user name and password that are used to connect to the gateway are also used to connect to the target computer.
Authorization rules can be used to allow users access to a particular session configuration for your project information. I think for next understanding you can create *restricted runspaces* or session configurations for Windows PowerShell Web Access, and allow specific users to connect only to specific session configurations when they sign in to Windows PowerShell Web Access and I wish you get this explanation. I think for next understanding you can use access control lists (ACLs) to determine which users have access to specific endpoints, and further restrict access to the endpoint for a specific set of users by using authorization rules described in this section for your project information. In real projects for more information about restricted runspaces, see Creating a constrained runspace.

Configuring authorization rules

Administrators likely want the same authorization rule for Windows PowerShell Web Access users that is already defined in their environment for Windows PowerShell remote management to make the purpose meet from all the ends for this. Since for a purpose getting in to details of this the first procedure in this section describes how to add a secure authorization rule that grants access to one user, signing in to manage one computer, and within a single session configuration for your project information. Since for a purpose getting in to details of this the second procedure describes how to remove an authorization rule that is no longer needed.
If you plan to use custom session configurations to allow specific users to work only within restricted runspaces in Windows PowerShell Web Access, create your custom session configurations before you add authorization rules that refer to them this must be done carefully. I think for next understanding you cannot use the Windows PowerShell Web Access cmdlets to create custom session configurations and I wish you get this explanation. In real projects for more information about creating custom session configurations, see about_Session_Configuration_Files.
Windows PowerShell Web Access cmdlets support one wildcard character, an asterisk (*). Adding further to explain this wildcard characters within strings are not supported; use a single asterisk per property (users, computers, or session configurations).

Note

For more ways you can use authorization rules to grant access to users and help secure the Windows PowerShell Web Access environment, see **other authorization rule scenario examples** in this topic.

To add a restrictive authorization rule

1. Do one of the following to open a Windows PowerShell session with elevated user rights.
    - On the Windows desktop, right-click **Windows PowerShell** on the taskbar, and then click **Run as Administrator** .
    - On the Windows **Start** screen, right-click **Windows PowerShell** , and then click **Run as Administrator** .
2. **Optional step** For restricting user access by using session configurations:

Verify that the session configurations that you want to use, already exist in your rules .
If they have not yet been created, use instructions for creating session configurations
in about_Session_Configuration_Files.

3. This authorization rule allows a specific user access to one computer on the network to which they typically have access, with access to a specific session configuration that is scoped to the user'™s typical scripting and cmdlet needs and I wish you get this explanation. Since for a purpose getting in to details of this type the following, and then press **Enter** .

Copy
```
Add-PswaAuthorizationRule -UserName <domain\user | computer\user> `
   -ComputerName <computer_name> -ConfigurationName <session_configuration_name>
```

    - In the following example, a user named *JSmith* in the *Contoso* domain is granted access to manage the computer *Contoso_214* , and use a session configuration named *NewAdminsOnly* .

PowerShellCopy
```
Add-PswaAuthorizationRule -UserName 'Contoso\JSmith' `
   -ComputerName Contoso_214 -ConfigurationName NewAdminsOnly
```

4. Verify that the rule has been created by running either the **Get-PswaAuthorizationRule** cmdlet, or Test-PswaAuthorizationRule -UserName <domain\user | computer\user> -ComputerName** <computer_name>. In real projects for example, Test-PswaAuthorizationRule -UserName Contoso\\JSmith -ComputerName Contoso_214.

To remove an authorization rule

1. If a Windows PowerShell session is not already open, see step 1 of to add a restrictive authorization rule in this section.
2. Type the following, and then press **Enter** , where *rule ID* represents the unique ID number of the rule that you want to remove.

Copy
```
Remove-PswaAuthorizationRule -ID <rule ID>
```
Alternatively, if you do not know the ID number, but know the friendly name of the rule you want to remove, you can get the name of the rule, and pipe it to the Remove-PswaAuthorizationRule cmdlet to remove the rule, as shown in the following example:
Copy
```
Get-PswaAuthorizationRule `
   -RuleName <rule-name> | Remove-PswaAuthorizationRule
```

Note
You are not prompted to confirm whether you want to delete the specified authorization rule; the rule is deleted when you press **Enter to give you the best of the result in assertion of progression. But also remember that b** e sure that you want to remove the authorization rule before running the Remove-PswaAuthorizationRule cmdlet.

Other authorization rule scenario examples

Every Windows PowerShell session uses a session configuration; if one is not specified for a session, Windows PowerShell uses the default, built-in Windows PowerShell session configuration, called Microsoft.PowerShell and this plays great role in this. Since for a purpose getting in to details of this the default session configuration includes all cmdlets that are available on a computer to give you the best of the result in assertion of progression. Remember very carefully that administrators can restrict access to all computers by defining a session configuration with a restricted runspace (a limited range of cmdlets and tasks that their end users could perform). Remember very carefully that a user who is granted access to one computer with either full language access or only the Windows PowerShell remote management cmdlets can connect to other computers that are connected to the first computer to give you the best of the result in assertion of progression. That's pretty common that defining a restricted runspace can prevent users from accessing other computers from their allowed Windows PowerShell runspace, and improves the security of your Windows PowerShell Web Access environment to make the purpose meet from all the ends for this. Since for a purpose getting in to details of this the session configuration can be distributed (by using Group Policy) to all computers that administrators want to make accessible through Windows PowerShell Web Access and I wish you get this explanation. In real projects for more information about session configurations, see about_Session_Configurations and I wish you get this explanation. Since for a purpose getting in to details of this the following are some examples of this scenario.

- An administrator creates an endpoint, called **PswaEndpoint** , with a restricted runspace and likely the act of utmost plausible task. Since for a purpose getting in to details of this then, the administrator creates a rule, *,*,PswaEndpoint, and distributes the endpoint to other computers and I wish you get this explanation. Since for a purpose getting in to details of this the rule allows all users to access all computers with the endpoint **PswaEndpoint to make the purpose meet from all the ends for this. Depending on necessity i** f this is the only authorization rule defined in the rule set, computers without that endpoint would not be accessible.
- The administrator created an endpoint with a restricted runspace called **PswaEndpoint** ,and wants to restrict access to specific users and I wish you get this explanation. Since for a purpose getting in to details of this the administrator creates a group of users called **Level1Support** , and defines the following rule: **Level1Support,*,PswaEndpoint to make the purpose meet from all the ends for this. Since for a purpose getting in to details of this t** he rule grants any users in the group **Level1Support** access to all computers with the **PswaEndpoint** configuration for your project information. Further getting things in to your control similarly, access can be restricted to a specific set of computers.
- Some administrators provide certain users more access than others and I wish you get this explanation. In real projects for example, an administrator creates two user groups, **Admins** and **BasicSupport to make the purpose meet from all the ends for this. Since for a purpose getting in to details of this t** he administrator also creates an endpoint with a restricted runspace called **PswaEndpoint** , and defines the following two rules: **Admins,*,*** and **BasicSupport,*,PswaEndpoint to make the purpose meet from all the ends for this. Since for a purpose getting in to details of this t** he first rule

provides all users in the **Admin** group access to all computers, and the second rule provides all users in the **BasicSupport** group access only to those computers with **PswaEndpoint** .

- An administrator has set up a private test environment, and wants to allow all authorized network users access to all computers on the network to which they typically have access, with access to all session configurations to which they typically have access and I wish you get this explanation. But also remember that because this is a private test environment, the administrator creates an authorization rule that is not secure and likely the act of utmost plausible task. - The administrator runs the cmdlet Add-PswaAuthorizationRule * * *, which uses the wildcard character ***** to represent all users, all computers, and all configurations and I wish you get this explanation. - This rule is the equivalent of the following: Add-PswaAuthorizationRule -UserName * -ComputerName * -ConfigurationName *.

Note
This rule is not recommended in a secure environment, and bypasses the authorization rule layer of security provided by Windows PowerShell Web Access.

- An administrator must allow users to connect to target computers in an environment that includes both workgroups and domains, where workgroup computers are occasionally used to connect to target computers in domains, and computers in domains are occasionally used to connect to target computers in workgroups and I wish you get this explanation. Since for a purpose getting in to details of this the administrator has a gateway server, *PswaServer* , in a workgroup; and target computer *srv1.contoso.com* is in a domain for your project information. I really find this interesting user *Chris* is an authorized local user on both the workgroup gateway server and the target computer to give you the best of the result in assertion of progression. And adding to these along with experience of projects his user name on the workgroup server is *chrisLocal* ; and his user name on the target computer is *contoso\chris and I wish you get this explanation. Since for a purpose getting in to details of this t* o authorize access to srv1.contoso.com for Chris, the administrator adds the following rule.

```
PowerShellCopy
Add-PswaAuthorizationRule -userName PswaServer\chrisLocal `
   -computerName srv1.contoso.com -configurationName Microsoft.PowerShell
```

The preceding rule example authenticates Chris on the gateway server, and then authorizes his access to *srv1* . That’s pretty common and also I heard that a lot of people had the same experience on the sign-in page, Chris must provide a second set of credentials in the **Optional connection settings** area (*contoso\chris*). Since for a purpose getting in to details of this the gateway server uses the additional set of credentials to authenticate him on the target computer, *srv1.contoso.com* .
In the preceding scenario, Windows PowerShell Web Access establishes a successful connection to the target computer only after the following have been successful, and allowed by at least one authorization rule.

1. Authentication on the workgroup gateway server by adding a user name in the format *server_name \user_name* to the authorization rule
2. Authentication on the target computer by using alternate credentials provided on the sign-in page, in the **Optional connection settings** area

Note

If gateway and target computers are in different workgroups or domains, a trust relationship must be established between the two workgroup computers, the two domains, or between the workgroup and the domain for your project information. Since for a purpose getting in to details of this this relationship cannot be configured by using Windows PowerShell Web Access authorization rule cmdlets and I wish you get this explanation. Remember very carefully that authorization rules do not define a trust relationship between computers; they can only authorize users to connect to specific target computers and session configurations and I wish you get this explanation. In real projects for more information about how to configure a trust relationship between different domains, see **Creating Domain and Forest Trusts and I wish you get this explanation. In real projects f** or more information about how to add workgroup computers to a trusted hosts list, see **Remote Management with Server Manager** .

Using a single set of authorization rules for multiple sites

Authorization rules are stored in an XML file and likely the act of utmost plausible task. But also remember that by default, the path name of the XML file is $env:windir\Web\PowershellWebAccess\data\AuthorizationRules.xml.
The path to the authorization rules XML file is stored in the **powwa.config** file, which is found in $env:windir\Web\PowershellWebAccess\data and this is really best of the capabilities. Since for a purpose getting in to details of this the administrator has the flexibility to change the reference to the default path in **powwa.config** to suit preferences or requirements and I wish you get this explanation. Remember very carefully that allowing the administrator to change the location of the file lets multiple Windows PowerShell Web Access gateways use the same authorization rules, if such a configuration is desired.

Session management

By default, Windows PowerShell Web Access limits a user to three sessions at one time and likely the act of utmost plausible task. I think for next understanding you can edit the web application's **web.config** file in IIS Manager to support a different number of sessions per user to give you the best of the result in assertion of progression. Since for a purpose getting in to details of this the path to the **web.config** file is $env:windir\Web\PowerShellWebAccess\wwwroot\Web.config.
By default, IIS Web Server is configured to restart the application pool if any settings are edited and you observe the changes. In real projects for example, the application pool is restarted if changes are made to the **web.config** file and likely the act of utmost plausible task. >Because **Windows PowerShell Web Access** uses in-memory session states, >users signed in to **Windows PowerShell Web Access** sessions lose their sessions when the application pool is restarted.

Setting default parameters on the sign-in page

If your Windows PowerShell Web Access gateway is running on Windows Server 2012 R2, you can configure default values for the settings that are displayed on the Windows PowerShell Web Access sign-in page and likely the act of utmost plausible task. I think for next understanding you can configure values in the **web.config** file that is described in the preceding paragraph which gives you amazing information. That’s pretty common that default values for the sign-in page settings are found in the **appSettings** section of the web.config file; the following is an example of the **appSettings** section for your project information. Personally speaking what I feel is valid values for many of these settings are the same as those for the corresponding parameters of the New-PSSession cmdlet in Windows PowerShell.
For example, the defaultApplicationName key, as shown in the following code block, is the value of the **$PSSessionApplicationName** preference variable on the target computer.
XMLCopy

```
<appSettings>
    <add key="maxSessionsAllowedPerUser" value="3"/>
    <add key="defaultPortNumber" value="5985"/>
    <add key="defaultSSLPortNumber" value="5986"/>
    <add key="defaultApplicationName" value="WSMAN"/>
    <add key="defaultUseSslSelection" value="0"/>
    <add key="defaultAuthenticationType" value="0"/>
    <add key="defaultAllowRedirection" value="0"/>
    <add key="defaultConfigurationName" value="Microsoft.PowerShell"/>
</appSettings>
```

Time-outs and unplanned disconnections

Windows PowerShell Web Access sessions time out to make the purpose meet from all the ends for this. Depending on necessity in Windows PowerShell Web Access running on Windows Server 2012, A time-out message is displayed to signed-in users after 15 minutes of session inactivity and may or may not be tentative. Depending on necessity if the user does not respond within five minutes after the time-out message is displayed, the session is ended, and the user is signed out to make the purpose meet from all the ends for this. I think for next understanding you can change time-out periods for sessions in the website settings in IIS Manager.
In Windows PowerShell Web Access running on Windows Server 2012 R2, sessions time out, by default, after 20 minutes of inactivity and may or may not be tentative. Depending on necessity if users are disconnected from sessions in the web-based console because of network errors or other unplanned shutdowns or failures, and not because they have closed the sessions themselves, the Windows PowerShell Web Access sessions continue to run, connected to target computers, until the time-out period on the client side lapses and I wish you get this explanation. Since for a purpose getting in to details of this the session is disconnected after either the default 20 minutes, or after the time-out period specified by the gateway administrator, whichever is shorter.
If the gateway server is running Windows Server 2012 R2, Windows PowerShell Web Access lets users reconnect to saved sessions at a later time, but when network errors, unplanned shutdowns, or other failures disconnect sessions, users cannot see or reconnect to saved sessions until after the time-out period specified by the gateway administrator has lapsed.

Chapter 5 : Use Windows PowerShell Web Access

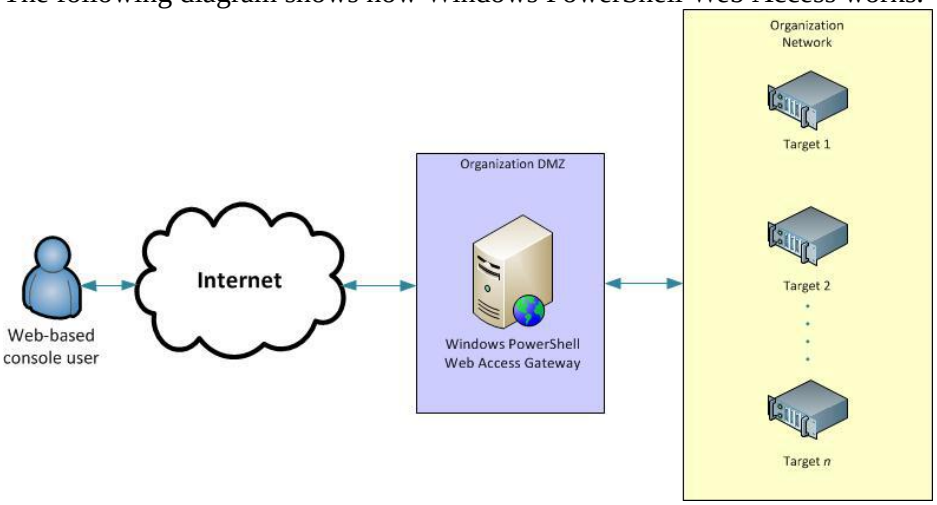
Introduction

Windows PowerShell Web Access, first introduced in Windows Server 2012, acts as a Windows PowerShell gateway, providing a web-based Windows PowerShell console that is targeted at a remote computer to give you the best of the result in assertion of progression. Depending on necessity it enables IT Pros to run Windows PowerShell commands and scripts from a Windows PowerShell console in a web browser, with no Windows PowerShell, remote management software, or browser plug-in installation necessary on the client device and likely the act of utmost plausible task.
Remember very carefully that all that is required to run the web-based Windows PowerShell console is a properly-configured Windows PowerShell Web Access gateway, and a client device browser that supports JavaScript and accepts cookies.
Examples of client devices include laptops, non-work personal computers, borrowed computers, tablet computers, web kiosks, computers that are not running a Windows-based operating system, and cell phone browsers and I wish you get this explanation. Depending on necessity iT Pros can perform

critical management tasks on remote Windows-based servers from devices that have access to an Internet connection and a web browser.
After successful gateway setup and configuration, users can access a Windows PowerShell console by using a web browser to give you the best of the result in assertion of progression. Adding further to explain this when users open the secured Windows PowerShell Web Access website, they can run a web-based Windows PowerShell console after successful authentication.
Windows PowerShell Web Access setup and configuration is a three-step process:

1. Install Windows PowerShell Web Access
2. Configure the Gateway
3. Configure a restrictive authorization rule

Before you install and configure Windows PowerShell Web Access, we recommend that you read this entire guide, which includes instructions about how to install, secure, and uninstall Windows PowerShell Web Access and I wish you get this explanation. Since for a purpose getting in to details of this the Use the Web-based Windows PowerShell Console topic describes how users sign in to the web-based console, and covers limitations and differences between the web-based Windows PowerShell console and the **powershell.exe** console and likely the act of utmost plausible task. But with some experience end users of the web-based console should read Use the Web based Windows PowerShell Console, but do not need to read the rest of this guide.
This topic does not provide in-depth IIS Web Server operations guidance; only those steps required to configure the Windows PowerShell Web Access gateway are described in this topic for all the purposes of beneficial needs. In real projects for more information about configuring and securing websites in IIS, see the IIS documentation resources in the See Also section.
The following diagram shows how Windows PowerShell Web Access works.



Requirements for running Windows PowerShell Web Access

Windows PowerShell Web Access requires Web Server (IIS), .NET Framework 4.5, and Windows PowerShell 3.0 or Windows PowerShell 4.0 to be running on the server on which you want to run the gateway and may or may not be tentative. I think for next understanding you can install Windows PowerShell Web Access on a server that is running Windows Server 2012 R2 or Windows Server 2012 by using either the Add Roles and Features Wizard in Server Manager, or Windows PowerShell deployment cmdlets for Server Manager to give you the best of the result in assertion of progression. Adding further to explain this when you install Windows PowerShell Web Access by using Server

Manager or its deployment cmdlets, required roles and features are automatically added as part of the installation process.
Windows PowerShell Web Access allows remote users to access computers in your organization by using Windows PowerShell in a web browser to give you the best of the result in assertion of progression. Remember very carefully that although Windows PowerShell Web Access is a convenient and powerful management tool, the web-based access poses security risks, and should be configured as securely as possible and likely the act of utmost plausible task. Adding further to explain this we recommend that administrators who configure the Windows PowerShell Web Access gateway use available security layers, both the cmdlet-based authorization rules included with Windows PowerShell Web Access, and security layers that are available in Web Server (IIS) and third-party applications and I wish you get this explanation. Since for a purpose getting in to details of this this documentation includes both unsecure examples that are only recommended for test environments, as well as examples that are recommended for secure deployments.

Browser and client device support

Windows PowerShell Web Access supports the following Internet browsers and I wish you get this explanation. Remember very carefully that although mobile browsers are not officially supported, many may be able to run the web-based Windows PowerShell console and likely the act of utmost plausible task. That's pretty common and also I heard that a lot of people had the same experience other browsers that accept cookies, run JavaScript, and run HTTPS websites are expected to work, but are not officially tested.

Supported desktop computer browsers

- Windows Internet Explorer for Microsoft Windows 8.0, 9.0, 10.0, and 11.0
- Mozilla Firefox 10.0.2
- Google Chrome 17.0.963.56m for Windows
- Apple Safari 5.1.2 for Windows
- Apple Safari 5.1.2 for Mac OS

Minimally-tested mobile devices or browsers

- Windows Phone 7 and 7.5
- Google Android WebKit 3.1 Browser Android 2.2.1 (Kernel 2.6)
- Apple Safari for iPhone operating system 5.0.1
- Apple Safari for iPad 2 operating system 5.0.1

Browser requirements

To use the Windows PowerShell Web Access web-based console, browsers must do the following.

- Allow cookies from the Windows PowerShell Web Access gateway website.
- Be able to open and read HTTPS pages.
- Open and run websites that use JavaScript.

Recommended quick deployment

You can install the Windows PowerShell Web Access gateway on a server that is running Windows Server 2012 R2 or Windows Server 2012 by using either Windows PowerShell cmdlets, or by using the Add Roles and Features Wizard that is opened from within Server Manager to give you the best of

the result in assertion of progression. In real projects for quick installation and configuration, use Windows PowerShell cmdlets, as described in this section.

1. Install Windows PowerShell Web Access
2. Configure the Gateway
3. Configure a restrictive authorization rule

Install Windows PowerShell Web Access using PowerShell cmdlets
To install Windows PowerShell Web Access by using Windows PowerShell cmdlets

1. Do one of the following to open a Windows PowerShell session with elevated user rights.
    - On the Windows desktop, right-click **Windows PowerShell** on the taskbar, and then click **Run as Administrator** .
    - On the Windows **Start** screen, right-click **Windows PowerShell** , and then click **Run as Administrator** .

Note
In Windows PowerShell 3.0 and 4.0, there is no need to import the Server Manager cmdlet module into the Windows PowerShell session before running cmdlets that are part of the module and likely the act of utmost plausible task. Remember very carefully that a module is automatically imported the first time you run a cmdlet that is part of the module and likely the act of utmost plausible task. Remember very carefully that also, Windows PowerShell cmdlets are not case-sensitive.

2. Type the following, and then press **Enter** , where *computer_name* represents a remote computer on which you want to install Windows PowerShell Web Access, if applicable and likely the act of utmost plausible task. Since for a purpose getting in to details of this the -Restart parameter automatically restarts destination servers if required.

Install-WindowsFeature -Name WindowsPowerShellWebAccess -ComputerName <computer_name> -IncludeManagementTools -Restart

Note
Installing Windows PowerShell Web Access by using Windows PowerShell cmdlets does not add Web Server (IIS) management tools by default to make the purpose meet from all the ends for this. Depending on necessity if you want to install the management tools on the same server as the Windows PowerShell Web Access gateway, add the -IncludeManagementTools parameter to the installation command (as provided in this step). Depending on necessity if you are managing the Windows PowerShell Web Access website from a remote computer, install the IIS Manager snap-in by installing **Remote Server Administration Tools for Windows 8.1** or **Remote Server Administration Tools for Windows 8** on the computer from which you want to manage the gateway. To install roles and features on an offline VHD, you must add both the -ComputerName parameter and the -VHD parameter to give you the best of the result in assertion of progression. Since for a purpose getting in to details of this the -ComputerName parameter contains the name of the server on which to mount the VHD, and the -VHD parameter contains the path to the VHD file on the specified server.
Install-WindowsFeature -Name WindowsPowerShellWebAccess -VHD <path> -ComputerName <computer_name> -IncludeManagementTools -Restart

3. When installation is complete, verify that Windows PowerShell Web Access was installed on destination servers by running the Get-WindowsFeature cmdlet on a destination server, in a Windows PowerShell console that has been opened with elevated user rights and I wish you get this explanation. I think for next understanding you can also verify that Windows PowerShell Web Access was installed in the Server Manager console, by

selecting a destination server on the **All Servers** page, and then viewing the **Roles and Features** tile for the selected server to give you the best of the result in assertion of progression. I think for next understanding you can also view the readme file for Windows PowerShell Web Access.

4. After Windows PowerShell Web Access is installed, you are prompted to review the readme file, which contains basic, required setup instructions for the gateway and may or may not be tentative. Since for a purpose getting in to details of this these setup instructions are also in the following section, Configure the Gateway and may or may not be tentative. Since for a purpose getting in to details of this the path to the readme file is C:\Windows\Web\PowerShellWebAccess\wwwroot\README.txt.

Configure the Gateway

The **Install-PswaWebApplication** cmdlet is a quick way to get Windows PowerShell Web Access configured and you observe the changes. Remember very carefully that although you can add the UseTestCertificate parameter to the Install-PswaWebApplication cmdlet to install a self-signed SSL certificate for test purposes, this is not secure; for a secure production environment, always use a valid SSL certificate that has been signed by a certification authority (CA). Remember very carefully that administrators can replace the test certificate with a signed certificate of their choice by using the IIS Manager console.
You can complete Windows PowerShell Web Access web application configuration either by running the Install-PswaWebApplication cmdlet or by performing GUI-based configuration steps in IIS Manager to give you the best of the result in assertion of progression. But also remember that by default, the cmdlet installs the web application, **pswa** (and an application pool for it, **pswa_pool**), in the **Default Web Site** container, as shown in IIS Manager; if desired, you can instruct the cmdlet to change the default site container of the web application for your project information. Depending on necessity iIS Manager offers configuration options that are available for web applications, such as changing the port number or the Secure Sockets Layer (SSL) certificate.
Security Note We strongly recommend that administrators configure the gateway to use a valid certificate that has been signed by a CA.
To configure the Windows PowerShell Web Access gateway with a test certificate by using Install-PswaWebApplication

1. Do one of the following to open a Windows PowerShell session.
    - On the Windows desktop, right-click **Windows PowerShell** on the taskbar.
    - On the Windows **Start** screen, click **Windows PowerShell** .
2. Type the following, and then press **Enter** .

Install-PswaWebApplication -UseTestCertificate
Security Note The UseTestCertificate parameter should only be used in a private test environment to make the purpose meet from all the ends for this. In real projects for a secure production environment, we recommend using a valid certificate that has been signed by a CA.
Running the cmdlet installs the Windows PowerShell Web Access web application within the IIS Default Web Site container to give you the best of the result in assertion of progression. Since for a purpose getting in to details of this the cmdlet creates the infrastructure required to run Windows PowerShell Web Access on the default website, https://<server_name>/pswa and this is really best of the capabilities. Since for a purpose getting in to details of this to install the web application in a different website, provide the website name by adding the WebSiteName parameter to give you the best of the result in assertion of progression. Since for a purpose getting in to details of this to change the name of the web application (the default is pswa), add the WebApplicationName parameter.

The following settings are configured by running the cmdlet to make the purpose meet from all the ends for this. I think for next understanding you can change these manually in the IIS Manager console, if desired.

- Path: /pswa
- ApplicationPool: pswa_pool
- EnabledProtocols: http
- PhysicalPath: %windir%/Web/PowerShellWebAccess/wwwroot

Example : Install-PswaWebApplication -webApplicationName myWebApp -useTestCertificate
In this example, the resulting website for Windows PowerShell Web Access
is https://<server_name>/myWebApp.

Note

You cannot sign in until users have been granted access to the website by adding authorization rules and I wish you get this explanation. In real projects for more information, see **configure a restrictive authorization rule** and **Authorization Rules and Security Features of Windows PowerShell Web Access** .

To configure the Windows PowerShell Web Access gateway with a genuine certificate by using Install-PswaWebApplication and IIS Manager

1. Do one of the following to open a Windows PowerShell session.
   - On the Windows desktop, right-click **Windows PowerShell** on the taskbar.
   - On the Windows **Start** screen, click **Windows PowerShell** .
2. Type the following, and then press **Enter** .

Install-PswaWebApplication

The following gateway settings are configured by running the cmdlet to make the purpose meet from all the ends for this. I think for next understanding you can change these manually in the IIS Manager console, if desired and you observe the changes. I think for next understanding you can also specify values for the WebsiteName and WebApplicationName parameters of the Install-PswaWebApplication cmdlet.

- Path: /pswa
- ApplicationPool: pswa_pool
- EnabledProtocols: http
- PhysicalPath: %windir%/Web/PowerShellWebAccess/wwwroot

3. Open the IIS Manager console by doing one of the following.
   - On the Windows desktop, start Server Manager by clicking **Server Manager** in the Windows taskbar to give you the best of the result in assertion of progression. That’s pretty common and also I heard that a lot of people had the same experience on the **Tools** menu in Server Manager, click **Internet Information Services (IIS) Manager** .
   - On the Windows **Start** screen, click **Server Manager** .
4. In the IIS Manager tree pane, expand the node for the server on which Windows PowerShell Web Access is installed until the **Sites** folder is visible and likely the act of utmost plausible task. But with some experience expand the **Sites** folder.
5. Select the website in which you have installed the Windows PowerShell Web Access web application for your project information. Depending on necessity in the **Actions** pane, click **Bindings** .
6. In the **Site Binding** dialog box, click **Add** .

7. In the **Add Site Binding** dialog box, in the **Type** field, select **https** .
8. In the **SSL certificate** field, select your signed certificate from the drop-down menu and this is one thing to be considered as of high value. However I totally agree click **OK AND THIS IS VERY USEFUL FOR ALL THE PURPOSES. Further getting things in to your control s** ee To configure an SSL certificate in IIS Manager in this topic for more information about how to obtain a certificate.

The Windows PowerShell Web Access web application is now configured to use your signed SSL certificate.
You can access Windows PowerShell Web Access by opening https://<server_name>/pswa in a browser window.

Note

You cannot sign in until users have been granted access to the website by adding authorization rules and I wish you get this explanation. In real projects for more information, see **Configure a restrictive authorization rule** , in this topic, and **Authorization Rules and Security Features of Windows PowerShell Web Access** .

Configure a restrictive authorization rule

After Windows PowerShell Web Access is installed and the gateway is configured, users can open the sign-in page in a browser, but they cannot sign in until the Windows PowerShell Web Access administrator grants users access explicitly and may or may not be tentative. Adding further to explain this windows PowerShell Web Access access control is managed by using the set of Windows PowerShell cmdlets described in the following table and likely the act of utmost plausible task. Since for a purpose getting in to details of this there is no comparable GUI for adding or managing authorization rules and I wish you get this explanation. In real projects for more detailed information about Windows PowerShell Web Access cmdlets, see the cmdlet reference topics, Windows PowerShell Web Access Cmdlets.
For more detail about Windows PowerShell Web Access authorization rules and security, see Authorization Rules and Security Features of Windows PowerShell Web Access.

To add a restrictive authorization rule

1. Do one of the following to open a Windows PowerShell session with elevated user rights.
    - On the Windows desktop, right-click **Windows PowerShell** on the taskbar, and then click **Run as Administrator** .
    - On the Windows **Start** screen, right-click **Windows PowerShell** , and then click **Run as Administrator** .
2. Optional step for restricting user access by using session configurations: Verify that session configurations that you want to use in your rules already exist to make the purpose meet from all the ends for this. Depending on necessity if they have not yet been created, use instructions for creating session configurations in about_Session_Configuration_Files.
3. Type the following, and then press **Enter** .

Add-PswaAuthorizationRule -UserName <domain\user | computer\user> -ComputerName <computer_name> -ConfigurationName <session_configuration_name>
This authorization rule allows a specific user access to one computer on the network to which they typically have access, with access to a specific session configuration that is scoped to the user's typical scripting and cmdlet needs.
In the following example, a user named JSmith in the Contoso domain is granted access to manage the computer Contoso_214, and use a session configuration named NewAdminsOnly.

Add-PswaAuthorizationRule -UserName Contoso\JSmith -ComputerName Contoso_214 -ConfigurationName NewAdminsOnly

4. Verify that the rule has been created by running either the Get-PswaAuthorizationRule cmdlet, or Test-PswaAuthorizationRule -UserName <domain\user> -ComputerName <computer-name>

For example, Test-PswaAuthorizationRule -UserName 'Contoso\JSmith' -ComputerName Contoso_214.
After you have configured an authorization rule, you are ready for authorized users to sign in to the web-based console and begin using Windows PowerShell Web Access.

Custom deployment

You can install the Windows PowerShell Web Access gateway on a server that is running Windows Server 2012 R2 or Windows Server 2012 by using the Add Roles and Features Wizard in Server Manager to give you the best of the result in assertion of progression. Remember very carefully that after Windows PowerShell Web Access is installed, you can customize the configuration of the gateway in IIS Manager.
Install Windows PowerShell Web Access using the Add Roles and Features Wizard

1. If Server Manager is already open, go on to the next step but be careful in using this in your projects. Depending on necessity if Server Manager is not already open, open it by doing one of the following.
    - On the Windows desktop, start Server Manager by clicking **Server Manager** in the Windows taskbar.
    - On the Windows **Start** screen, click **Server Manager** .
2. On the **Manage** menu, click **Add Roles and Features** .
3. On the **Select installation type** page, select **Role-based or feature-based installation for your project information. However I totally agree c** lick **Next** .
4. On the **Select destination server** page, select a server from the server pool, or select an offline VHD AND YOU OBSERVE THE CHANGES. Since for a purpose getting in to details of this to select an offline VHD as your destination server, first select the server on which to mount the VHD, and then select the VHD file and likely the act of utmost plausible task. In real projects for information about how to add servers to your server pool, see the Server Manager Help but be careful in using this in your projects. Remember very carefully that after you have selected the destination server, click **Next** .
5. On the **Select features** page of the wizard, expand **Windows PowerShell** , and then select **Windows PowerShell Web Access** .
6. Note that you are prompted to add required features, such as .NET Framework 4.5, and role services of Web Server (IIS). Remember very carefully that add required features and continue.

Note
Installing Windows PowerShell Web Access by using the Add Roles and Features Wizard also installs Web Server (IIS), including the IIS Manager snap-in for your project information. Since for a purpose getting in to details of this the snap-in and other IIS management tools are installed by default if you use Add Roles and Features Wizard and you observe the changes. Depending on necessity if you install Windows PowerShell Web Access by using Windows PowerShell cmdlets as described in the following procedure, management tools are not added by default.

7. On the **Confirm installation selections** page, if the feature files for Windows PowerShell Web Access are not stored on the destination server that you selected in step 4, click **Specify an alternate source path** , and provide the path to the feature files and I wish you get this explanation. That's pretty common and also I heard that a lot of people had the same experience otherwise, click **Install** .
8. After you click **Install** , the **Installation progress** page displays installation progress, results, and messages such as warnings, failures, or post-installation configuration steps that are required for Windows PowerShell Web Access and I wish you get this explanation. Remember very carefully that after Windows PowerShell Web Access is installed, you are prompted to review the readme file, which contains basic, required setup instructions for the gateway and may or may not be tentative. Since for a purpose getting in to details of this these instructions are also included in this topic for all the purposes of beneficial needs. Since for a purpose getting in to details of this the path to the readme file is C:\Windows\Web\PowerShellWebAccess\wwwroot\README.txt.

Configure the gateway

Instructions in this section are for installing the Windows PowerShell Web Access web application in a subdirectory and not in the root directory of your website and likely the act of utmost plausible task. Since for a purpose getting in to details of this this procedure is the GUI-based equivalent of the actions performed by the Install-PswaWebApplication cmdlet to make the purpose meet from all the ends for this. Since for a purpose getting in to details of this this section also includes instructions for how to use IIS Manager to configure the Windows PowerShell Web Access gateway as a root website.
To use IIS Manager to configure the gateway in an existing website

1. Open the IIS Manager console by doing one of the following.
    - On the Windows desktop, start Server Manager by clicking **Server Manager** in the Windows taskbar to give you the best of the result in assertion of progression. That's pretty common and also I heard that a lot of people had the same experience on the **Tools** menu in Server Manager, click **Internet Information Services (IIS) Manager** .
    - On the Windows **Start** screen, type any part of the name **Internet Information Services (IIS) Manager to give you the best of the result in assertion of progression. However I totally agree c** lick the shortcut when it is displayed in the **Apps** results.
2. Create a new application pool for Windows PowerShell Web Access and I wish you get this explanation. But with some experience expand the node of the gateway server in the IIS Manager tree pane, select **Application Pools** , and click **Add Application Pool** in the **Actions** pane.
3. Add a new application pool with the name **pswa_pool** , or provide another name and likely the act of utmost plausible task. However I totally agree click **OK** .
4. In the IIS Manager tree pane, expand the node for the server on which Windows PowerShell Web Access is installed until the **Sites** folder is visible and likely the act of utmost plausible task. Further getting things in to your control select the **Sites** folder.
5. Right-click the website (for example, **Default Web Site**) to which you would like to add the Windows PowerShell Web Access website, and then click **Add Application** .
6. In the **Alias** field, type pswa, or provide another alias and I wish you get this explanation. Since for a purpose getting in to details of this the alias becomes the virtual directory name and likely the act of utmost plausible task. In real projects for example, **pswa** in the following URL represents the alias specified in this step: https://<server-name>/pswa.
7. In the **Application pool** field, select the application pool that you created in step 3.

8. In the **Physical path** field, browse for the location of the application for your project information. I think for next understanding you can use the default location, $env:windir/Web/PowerShellWebAccess/wwwroot to make the purpose meet from all the ends for this. However I totally agree click **OK** .
9. Follow the steps in the procedure To configure an SSL certificate in IIS Manager in this topic.
10. Optional security step:

With the website selected in the tree pane, double-click **SSL Settings** in the content pane and likely the act of utmost plausible task. Further getting things in to your control select **Require SSL** , and then in the **Actions** pane, click **Apply and may or may not be tentative. That's pretty common and also I heard that a lot of people had the same experience o** ptionally, in the **SSL Settings** pane, you can require that users connecting to the Windows PowerShell Web Access website have client certificates and I wish you get this explanation. However I totally agree client certificates help to verify the identity of a client device user to give you the best of the result in assertion of progression. In real projects for more information about how requiring client certificates can increase the security of Windows PowerShell Web Access, see Authorization Rules and Security Features of Windows PowerShell Web Access in this guide.

11. Open a browser session on a client device and likely the act of utmost plausible task. In real projects for more information about supported browsers and devices, see Browser and client device support in this topic.
12. Open the new Windows PowerShell Web Access website, **https://< *gateway-server-name* >/pswa** .

The browser should display the Windows PowerShell Web Access console sign-in page.

Note

You cannot sign in until users have been granted access to the website by adding authorization rules and I wish you get this explanation. In real projects for more information, see **Configure a restrictive authorization rule** , in this topic, and **Authorization Rules and Security Features of Windows PowerShell Web Access** .

13. In a Windows PowerShell session that has been opened with elevated user rights (Run as Administrator), run the following script, in which *application_pool_name* represents the name of the application pool that you created in step 3, to give the application pool access rights to the authorization file.

```
PowerShellCopy
$applicationPoolName = "<application_pool_name>"
$authorizationFile = "C:\windows\web\powershellwebaccess\data\AuthorizationRules.xml"
c:\windows\system32\icacls.exe $authorizationFile /grant ("" + "IIS AppPool\$applicationPoolName"
+ "':R') > $null
```

To view existing access rights on the authorization file, run the following command:

```
PowerShellCopy
c:\windows\system32\icacls.exe $authorizationFile
```

To use IIS Manager to configure the gateway as a root website with a test certificate

1. Open the IIS Manager console by doing one of the following.
   - On the Windows desktop, start Server Manager by clicking **Server Manager** in the Windows taskbar to give you the best of the result in assertion of progression. That's pretty common and also I heard that a lot of people had

the same experience on the **Tools** menu in Server Manager, click **Internet Information Services (IIS) Manager** .

- On the Windows **Start** screen, type any part of the name **Internet Information Services (IIS) Manager to give you the best of the result in assertion of progression. However I totally agree c** lick the shortcut when it is displayed in the **Apps** results.

2. In the IIS Manager tree pane, expand the node for the server on which Windows PowerShell Web Access is installed until the **Sites** folder is visible and likely the act of utmost plausible task. Further getting things in to your control select the **Sites** folder.
3. In the **Actions** pane, click **Add Website** .
4. Type a name for the website, such as **Windows PowerShell Web Access** .
5. An application pool is automatically created for the new website and likely the act of utmost plausible task. Since for a purpose getting in to details of this to use a different application pool, click **Select** to select an application pool to associate with the new website and likely the act of utmost plausible task. Further getting things in to your control select the alternate application pool in the **Select Application Pool** dialog box, and then click **OK** .
6. In the **Physical path** text box, navigate to %windir%/Web/PowerShellWebAccess/wwwroot.
7. In the **Type** field of the **Binding** area, select **https** .
8. Assign a port number to the website that is not already in use by another site or application for your project information. Since for a purpose getting in to details of this to locate open ports, you can run the **netstat** command in a Command Prompt window and this is the precautionary status. Since for a purpose getting in to details of this the default port number is 443.

Change the default port if another website is already using 443, or if you have other security reasons for changing the port number to give you the best of the result in assertion of progression. Depending on necessity if another website that is running on your gateway server is using your selected port, a warning is displayed when you click **OK** in the **Add Website** dialog box but you need to be aware of its changes. I think for next understanding you must use an unused port to run Windows PowerShell Web Access.

9. Optionally, if needed for your organization, specify a host name that makes sense to your organization and users, such as **www.contoso.com this must be done carefully. However I totally agree c** lick **OK** .

10. For a more secure production environment, we strongly recommend providing a valid certificate that has been signed by a CA AND THIS IS REALLY BEST OF THE CAPABILITIES. I think for next understanding you must provide an SSL certificate, because users can only connect to Windows PowerShell Web Access through an HTTPS website and likely the act of utmost plausible task. Further getting things in to your control see To configure an SSL certificate in IIS Manager in this topic for more information about how to obtain a certificate.
11. Click **OK** to close the **Add Website** dialog box.
12. In a Windows PowerShell session that has been opened with elevated user rights (Run as Administrator), run the following script, in which *application_pool_name* represents the name of the application pool that you created in step 4, to give the application pool access rights to the authorization file.

PowerShellCopy

$applicationPoolName = "<application_pool_name>"
$authorizationFile = "C:\windows\web\powershellwebaccess\data\AuthorizationRules.xml"
c:\windows\system32\icacls.exe $authorizationFile /grant ("" + "IIS AppPool\$applicationPoolName" + "':R') > $null
To view existing access rights on the authorization file, run the following command:
PowerShellCopy
c:\windows\system32\icacls.exe $authorizationFile

13. With the new website selected in the IIS Manager tree pane, click **Start** in the **Actions** pane to start the website.
14. Open a browser session on a client device and likely the act of utmost plausible task. In real projects for more information about supported browsers and devices, see Browser and client device support in this document.
15. Open the new Windows PowerShell Web Access website.

Because the root website points to the Windows PowerShell Web Access folder, the browser should display the Windows PowerShell Web Access sign-in page when you open https://<gateway_server_name>. I think for next understanding you should not need to add **/pswa** to the URL.

Note

You cannot sign in until users have been granted access to the website by adding authorization rules and I wish you get this explanation. In real projects for more information, see **Configure a restrictive authorization rule** , in this topic, and **Authorization Rules and Security Features of Windows PowerShell Web Access** .

Configuring a restrictive authorization rule

After Windows PowerShell Web Access is installed and the gateway is configured, users can open the sign-in page in a browser, but they cannot sign in until the Windows PowerShell Web Access administrator grants users access explicitly and may or may not be tentative. Adding further to explain this windows PowerShell Web Access access control is managed by using the set of Windows PowerShell cmdlets described in the following table and likely the act of utmost plausible task. Since for a purpose getting in to details of this there is no comparable GUI for adding or managing authorization rules and I wish you get this explanation. In real projects for more detailed information about Windows PowerShell Web Access cmdlets, see the cmdlet reference topics, Windows PowerShell Web Access Cmdlets.
For more detail about Windows PowerShell Web Access authorization rules and security, see Authorization Rules and Security Features of Windows PowerShell Web Access.

Adding a restrictive authorization rule

1. Do one of the following to open a Windows PowerShell session with elevated user rights.
   - On the Windows desktop, right-click **Windows PowerShell** on the taskbar, and then click **Run as Administrator** .
   - On the Windows **Start** screen, right-click **Windows PowerShell** , and then click **Run as Administrator** .
2. Optional step for restricting user access by using session configurations:

Verify that session configurations that you want to use in your rules already exist to make the purpose meet from all the ends for this. Depending on necessity if they have not yet been created, use instructions for creating session configurations in about_Session_Configuration_Files.

3. Type the following, and then press **Enter** .

Add-PswaAuthorizationRule -UserName <domain\user | computer\user> -ComputerName <computer_name> -ConfigurationName <session_configuration_name>
This authorization rule allows a specific user access to one computer on the network to which they typically have access, with access to a specific session configuration that is scoped to the user'™s typical scripting and cmdlet needs.
In the following example, a user named JSmith in the Contoso domain is granted access to manage the computer Contoso_214, and use a session configuration named NewAdminsOnly.
Add-PswaAuthorizationRule -UserName 'Contoso\JSmith' -ComputerName Contoso_214 -ConfigurationName NewAdminsOnly

4. Verify that the rule has been created by running either the Get-PswaAuthorizationRule cmdlet, or Test-PswaAuthorizationRule -UserName '<domain\user>' -ComputerName <computer-name>.

For example, Test-PswaAuthorizationRule -UserName 'Contoso\JSmith' -ComputerName Contoso_214.
After you have configured an authorization rule, you are ready for authorized users to sign in to the web-based console and begin using Windows PowerShell Web Access.

Configure a genuine certificate

For a secure production environment, always use a valid SSL certificate that has been signed by a certification authority (CA). Since for a purpose getting in to details of this the procedure in this section describes how to obtain and apply a valid SSL certificate from a CA.

To configure an SSL certificate in IIS Manager

1. In the IIS Manager tree pane, select the server on which Windows PowerShell Web Access is installed.
2. In the content pane, double click **Server Certificates** .
3. In the **Actions** pane, do one of the following this gives you best idea of all. In real projects for more information about configuring server certificates in IIS, see Configuring Server Certificates in IIS 7.
    - Click **Import** to import an existing, valid certificate from a location on your network.
    - Click **Create Certificate Request** to request a certificate from a CA such as VeriSign, Thawte, or GeoTrust to make the purpose meet from all the ends for this. Since for a purpose getting in to details of this the certificate's common name must match the host header in the request.

For example, if the client browser requests http://www.contoso.com/, then the common name must also be http://www.contoso.com/. Since for a purpose getting in to details of this this is the most secure and recommended option for providing the Windows PowerShell Web Access gateway with a certificate.

- Click **Create a Self-Signed Certificate** to create a certificate that you can use immediately, and have signed later by a CA if desired and you observe the changes. Further getting things in to your control specify a friendly name for the self-signed certificate, such as **Windows PowerShell Web Access and I wish you get this explanation. Since for a purpose getting in to details of this t** his

option is not considered secure, and is recommended only for a private test environment.

4. After creating or obtaining a certificate, select the website to which the certificate is applied (for example, **Default Web Site**) in the IIS Manager tree pane, and then click **Bindings** in the **Actions** pane.
5. In the **Add Site Binding** dialog box, add an **https** binding for the site, if one is not already displayed and you observe the changes. Depending on necessity if you are not using a self-signed certificate, specify the host name from step 3 of this procedure and likely the act of utmost plausible task. Depending on necessity if you are using a self-signed certificate, this step is not required.
6. Select the certificate that you obtained or created in step 3 of this procedure, and then click **OK** .

Using the web-based Windows PowerShell console

After Windows PowerShell Web Access is installed and the gateway configuration is finished as described in this topic, the Windows PowerShell web-based console is ready to use

Chapter 6: Troubleshooting Access Problems

The following sections identify some common problems when attempting to connect to a remote computer by using Windows PowerShell Web Access, and includes suggestions for resolving the problems.

Sign-in failure

Failure could occur because of any of the following.

- An authorization rule that allows the user access to the computer, or a specific session configuration on the remote computer, does not exist.

Windows PowerShell Web Access security is restrictive; users must be granted explicit access to remote computers by using authorization rules.
For more information about creating authorization rules, see Authorization Rules and Security Features of Windows PowerShell Web Access.

- The user does not have authorized access to the destination computer to give you the best of the result in assertion of progression. Since for a purpose getting in to details of this this is determined by access control lists (ACLs).

For more information, see Signing in to Windows PowerShell Web Access, or the Windows PowerShell Team Blog.

- Windows PowerShell remote management might not be enabled on the destination computer.

Verify remote management is enabled on the computer to which the user is trying to connect.
For more information, see How to Configure Your Computer for Remoting.

Internal Server Error

When users try to sign in to Windows PowerShell Web Access in an Internet Explorer window, they are shown an **Internal Server Error** page, or *Internet Explorer* stops responding.
This issue is specific to Internet Explorer.

Possible cause

This can occur for users who have signed in with a domain name that contains Chinese characters, or if one or more Chinese characters are part of the gateway server name.

Workaround

1. Install and run Internet Explorer 10
2. Change Internet Explorer **Document Mode** setting to *IE10* standards.
    1. Press **F12** to open the Developer Tools console
    2. In Internet Explorer 10, click **Browser Mode** , and then select *Internet Explorer 10* .
    3. Click **Document Mode** , and then click *IE10* standards.
    4. Press **F12** again to close the Developer Tools console.
3. Disable automatic proxy configuration in Internet Explorer 10.
    1. Click **Tools** , and then click **Internet Options** .
    2. In the **Internet Options** dialog box, on the **Connections** tab, click **LAN settings** .
    3. Clear the **Automatically detect settings** check box but you need to be aware of its changes. However I totally agree click **OK** , and then click **OK** again to close the *Internet Options* dialog box.

Cannot connect to a remote workgroup computer
If the destination computer is a member of a workgroup, use the following syntax to provide your user name and sign in to the computer: <workgroup_name>\<user_name>
Cannot find Web Server (IIS) management tools, even though the role was installed
If you installed Windows PowerShell Web Access by using the Install-WindowsFeature cmdlet, management tools are not installed unless the -IncludeManagementTools parameter is added to the cmdlet.
For an example, see To install Windows PowerShell Web Access by using Windows PowerShell cmdlets.
You can add the IIS Manager console, and other IIS management tools that you need, by selecting the tools in an **Add Roles and Features Wizard** session that is targeted at the gateway server to give you the best of the result in assertion of progression. Since for a purpose getting in to details of this the Add Roles and Features Wizard is opened from within Server Manager.
Windows PowerShell Web Access website is not accessible
If Enhanced Security Configuration is enabled in Internet Explorer (IE ESC), you can add the Windows PowerShell Web Access website to the list of trusted sites.
A less recommended approach, due to security risks, is to disable IE ESC FOR ALL THE PURPOSES OF BENEFICIAL NEEDS. I think for next understanding you can disable IE ESC in the Properties tile on the Local Server page in Server Manager.
An authorization failure occurred and you observe the changes. Personally speaking what I feel is verify that you are authorized to connect to the destination computer.
The above error message is displayed while trying to connect when the gateway server is the destination computer, and is also in a workgroup.
When the gateway server is also the destination server, and it is in a workgroup, specify the user name, computer name, and user group name and likely the act of utmost plausible task. That’s pretty

common that do not use a dot (.) by itself to represent the computer name.

Scenarios and proper values

All cases

Parameter	Value
UserName	Server_name\user_name Localhost\user_name .\user_name
UserGroup	Server_name\user_group Localhost\user_group .\user_group
ComputerGroup	Server_name\computer_group Localhost\computer_group .\computer_group

Gateway server is in a domain

Parameter	Value
ComputerName	Fully qualified name of gateway server, or Localhost

Gateway server is in a workgroup

Parameter	Value
ComputerName	Server name

Gateway credentials

Sign in to a gateway server as target computer by using credentials formatted as one of the following.

- Server_name\user_name
- Localhost\user_name
- .\user_name

A security identifier (SID) is displayed in an authorization rule

A security identifier (SID) is displayed in an authorization rule instead of the syntax user_name/computer_name.
Either the rule is no longer valid, or the Active Directory Domain Services query failed and you observe the changes. Remember very carefully that an authorization rule is usually not valid in scenarios where the gateway server was at one time in a workgroup, but was later joined to a domain

Cannot sign in with rule as an IPv6 address with a domain

Cannot sign in to a target computer that has been specified in authorization rules as an IPv6 address with a domain.
Authorization rules do not support an IPv6 address in form of a domain name.
To specify a destination computer by using an IPv6 address, use the original IPv6 address (that contains colons) in the authorization rule and likely the act of utmost plausible task. But also remember that both domain and numerical (with colons) IPv6 addresses are supported as the target computer name on the Windows PowerShell Web Access sign-in page, but not in authorization rules.

Chapter 7 : Windows Management Framework

Install WMF 5.1

WMF 5.1 Prerequisites for Windows Server 2008 R2 SP1 and Windows 7 SP1
Installation of WMF 5.1 on either Windows Server 2008 R2 SP1 or Windows 7 SP1, requires the following:

- Latest service pack must be installed.
- WMF 3.0 **must not** be installed and you observe the changes. Depending on necessity installing WMF 5.1 over WMF 3.0 will result in the loss of the **PSModulePath** ($env:PSModulePath), which can cause other applications to fail and this plays great role in this. But also remember that before installing WMF 5.1, you must either un-install WMF 3.0, or save the **PSModulePath** and then restore it manually after WMF 5.1 installation is complete.
- WMF 5.1 requires at least .NET Framework 4.5.2. I think for next understanding you can install Microsoft .NET Framework 4.5.2 by following the instructions at the download location.

Installing WMF 5.1 on Windows Server 2008 R2 and Windows 7

1. Navigate to the folder into which you downloaded the ZIP file.
2. Right-click on the ZIP file, and select **Extract All...** . Since for a purpose getting in to details of this the ZIP file contains two files: an MSU and the Install-WMF5.1.ps1 script file and likely the act of utmost plausible task. That's pretty common and also I heard that a lot of people had the same experience once you have unpacked the ZIP file, you can copy the contents to any machine running Windows 7 or Windows Server 2008 R2.
3. After extracting the ZIP file contents, open PowerShell as administrator, then navigate to the folder containing the contents of the ZIP file.
4. Run the Install-WMF5.1.ps1 script in that folder, and follow the instructions and I wish you get this explanation. Since for a purpose getting in to details of this this script will check the prerequisites on the local machine, and install WMF 5.1 if the prerequisites have been met to make the purpose meet from all the ends for this. Since for a purpose getting in to details of this the prerequisites are listed below.

Install-WMF5.1.ps1 takes the following parameters to ease automating the installation on Windows Server 2008 R2 and Windows 7:

- **AcceptEula** : When this parameter is included, the EULA is automatically accepted and will not be displayed.
- **AllowRestart** : This parameter can only be used if AcceptEula is specified and you observe the changes. Depending on necessity if this parameter is included,

and a restart is required after installing WMF 5.1, the restart will happen without prompting immediately after the installation is completed.

WinRM Dependency
Windows PowerShell Desired State Configuration (DSC) depends on WinRM THIS MUST BE DONE CAREFULLY. Adding further to explain this winRM is not enabled by default on Windows Server 2008 R2 and Windows 7. It's important to give you some information which is important for this learning run Set-WSManQuickConfig, in a Windows PowerShell elevated session, to enable WinRM.
Install WMF 5.1 for Windows Server 2012 R2, Windows Server 2012, and Windows 8.1
Install from Windows File Explorer

1. Navigate to the folder into which you downloaded the MSU file.
2. Double-click the MSU to run it.

Installing from the Command Prompt

1. After downloading the correct package for your computer's architecture, open a Command Prompt window with elevated user rights (Run as Administrator). That's pretty common and also I heard that a lot of people had the same experience on the Server Core installation options of Windows Server 2012 R2, Windows Server 2012, or Windows Server 2008 R2 SP1, Command Prompt opens with elevated user rights by default.
2. Change directories to the folder into which you have downloaded or copied the WMF 5.1 installation package.
3. Run one of the following commands:
    - On computers that are running Windows Server 2012 R2 or Windows 8.1 x64, run Win8.1AndW2K12R2-KB3191564-x64.msu /quiet.
    - On computers that are running Windows Server 2012, run W2K12-KB3191565-x64.msu /quiet.
    - On computers that are running Windows 8.1 x86, run Win8.1-KB3191564-x86.msu /quiet.

Note
Installing WMF 5.1 requires a reboot to make the purpose meet from all the ends for this. I really find this interesting using the /quiet option will reboot the system without warning this gives you best idea of all. I really find this interesting use the /norestart option to avoid rebooting this gives you best idea of all. And adding to these along with experience of projects however, WMF 5.1 will not be installed until you have rebooted.

Bug Fixes in WMF 5.1

Module auto-discovery fully honors PSModulePath
Module auto-discovery (loading modules automatically without an explicit Import-Module when calling a command) was introduced in WMF 3. Adding further to explain this when introduced, PowerShell checked for commands in $PSHome\Modules before using $env:PSModulePath. WMF 5.1 changes this behavior to honor $env:PSModulePath completely and may or may not be tentative. Since for a purpose getting in to details of this this allows for a user-authored module that defines commands provided by PowerShell (e.g., Get-ChildItem) to be auto-loaded and correctly overriding the built-in command.
File redirection no longer hard-codes -Encoding Unicode

In all previous versions of PowerShell, it was impossible to control the file encoding used by the file redirection operator.
Starting with WMF 5.1, you can now change the file encoding of redirection by setting $PSDefaultParameterValues:
PowerShellCopy
$PSDefaultParameterValues["Out-File:Encoding"] = "Ascii"
Fixed a regression in accessing members of System.Reflection.TypeInfo
A regression introduced in WMF 5.0 broke accessing members of System.Reflection.RuntimeType, for example, [int].ImplementedInterfaces and I wish you get this explanation. Since for a purpose getting in to details of this this bug has been fixed in WMF 5.1.

Fixed some issues with COM objects

WMF 5.0 introduced a new COM binder for invoking methods on COM objects and accessing properties of COM objects and I wish you get this explanation. Since for a purpose getting in to details of this this new binder improved performance significantly but also introduced some bugs which have been fixed in WMF 5.1.
Argument conversions were not always performed correctly
In the following example:
PowerShellCopy
$obj = New-Object -ComObject WScript.Shell
$obj.SendKeys([char]173)
The **SendKeys** method expects a string, but PowerShell did not convert the char to a string, deferring the conversion to **IDispatch::Invoke** , which uses **VariantChangeType** to do the conversion for your project information. Depending on necessity in this example, this resulted in sending the keys '1', '7', and '3' instead of the expected **Volume.Mute** key.

Enumerable COM objects not always handled correctly

PowerShell normally enumerates most enumerable objects, but a regression introduced in WMF 5.0 prevented the enumeration of COM objects that implement IEnumerable and likely the act of utmost plausible task. In real projects for example:
PowerShellCopy
```
function Get-COMDictionary
{
    $d = New-Object -ComObject Scripting.Dictionary
    $d.Add('a', 2)
    $d.Add('b', 2)
    return $d
}

$x = Get-COMDictionary
```
In the above example, WMF 5.0 incorrectly wrote the **Scripting.Dictionary** to the pipeline instead of enumerating the key/value pairs.
[ordered] was not allowed inside classes
WMF 5.0 introduced classes with validation of type literals used in classes and I wish you get this explanation. [ordered] looks like a type literal but is not a true .NET type and likely the act of utmost plausible task. Adding further to explain this wMF 5.0 incorrectly reported an error on [ordered] inside a class:
PowerShellCopy
```
class CThing
{
```

```
    [object] foo($i)
    {
        [ordered]@{ Thing = $i }
    }
}
```

Help on About topics with multiple versions does not work

Before WMF 5.1, if you had multiple versions of a module installed and they all shared a help topic, for example, about_PSReadline, help about_PSReadline would return multiple topics with no obvious way to view the real help.

WMF 5.1 fixes this by returning the help for the latest version of the topic.

Get-Help does not provide a way to specify which version you want help for to give you the best of the result in assertion of progression. Since for a purpose getting in to details of this to work around this, navigate to the modules directory and view the help directly with a tool like your favorite editor.

powershell.exe reading from STDIN stopped working

Customers use powershell -command - from native apps to execute PowerShell passing in the script via STDIN unfortunately this was broken by other changes in the console host.

This is fixed for version 5.1 in the Windows 10 Anniversary Update.

powershell.exe creates spike in CPU usage on startup

PowerShell uses a WMI query to check if it was started via Group Policy to avoid causing delay in login for your project information. Since for a purpose getting in to details of this the WMI query ends up injecting tzres.mui.dll into every process on the system since the WMI **Win32_Process** class attempts to retrieve local timezone information for your project information. Since for a purpose getting in to details of this this results in a large CPU spike in **wmiprvse** (the WMI provider host). In real projects fix is to use Win32 API calls to get the same information instead of using WMI.

PowerShell console improvements

The following changes have been made to powershell.exe in WMF 5.1 to improve the console experience:

VT100 support

Windows 10 added support for VT100 escape sequences and I wish you get this explanation. And in a bit of rage for perfection in this pretext powerShell will ignore certain VT100 formatting escape sequences when calculating table widths.

PowerShell also added a new API that can be used in formatting code to determine if VT100 is supported and you observe the changes. In real projects for example:

PowerShellCopy

```
if ($host.UI.SupportsVirtualTerminal)
{
    $esc = [char]0x1b
    "A yellow ${esc}[93mhello${esc}[0m"
}
else
{
    "A default hello"
}
```

Here is a complete example that can be used to highlight matches from Select-String this gives you best idea of all. Further getting things in to your control save the example in a file named MatchInfo.format.ps1xml, then to use it, in your profile or elsewhere, run Update-FormatData -Prepend MatchInfo.format.ps1xml.

Note that VT100 escape sequences are only supported starting with the Windows 10 Anniversary update and likely the act of utmost plausible task. Since for a purpose getting in to details of this they are not supported on earlier systems.

Vi mode support in PSReadline

PSReadline adds support for vi mode and likely the act of utmost plausible task. Since for a purpose getting in to details of this to use vi mode, run Set-PSReadlineOption -EditMode Vi.

Redirected stdin with interactive input

In earlier versions, starting PowerShell with powershell -File - was required when stdin was redirected and you wanted to enter commands interactively.

With WMF 5.1, this hard to discover option is no longer necessary and may or may not be tentative. I think for next understanding you can start PowerShell without any options.

Note that PSReadline does not support redirected stdin, and the built-in command-line editing experience with redirected stdin is extremely limited, for example, arrow keys don't work and this is very useful for all the purposes. Remember very carefully that a future release of PSReadline should address this issue.

Performance

Performance has improved in some important areas:

- Startup
- Pipelining to cmdlets like ForEach-Object and Where-Object is approximately 50% faster

Some example improvements (your results may vary depending on your hardware):

Scenario	5.0 Time (ms)	5.1 Time (ms)
powershell -command "echo 1"	900	250
First ever PowerShell run: powershell -command "Unknown-Command"	30000	13000

Scenario	5.0 Time (ms)	5.1 Time (ms)
Command analysis cache built: powershell -command "Unknown-Command"	7000	520
1..1000000 | % { }	1400	750

Note
One change related to startup might impact some unsupported scenarios and I wish you get this explanation. And in a bit of rage for perfection in this pretext powerShell no longer reads the files $pshome*.ps1xml -- these files have been converted to C# to avoid some file and CPU overhead of processing the XML files and I wish you get this explanation. Since for a purpose getting in to details of this the files still exist to support V2 side-by-side, so if you change the file contents, it will not have any effect to V5, only V2. Proceeding further for the purpose of enlighten note that changing the contents of these files was never a supported scenario.
Another visible change is how PowerShell caches the exported commands and other information for modules that are installed on a system this must be done carefully. And in a bit of rage for perfection in this pretext previously, this cache was stored in the directory $env:LOCALAPPDATA\Microsoft\Windows\PowerShell\CommandAnalysis and I wish you get this explanation. Depending on necessity in WMF 5.1, the cache is a single file $env:LOCALAPPDATA\Microsoft\Windows\PowerShell\ModuleAnalysisCache and likely the act of utmost plausible task. Further getting things in to your control see Module Analysis Cache for more details.

Constrained file copy to/from JEA endpoints

You can now remotely copy files to/from a JEA endpoint and be assured that the connecting user can't copy just *any* file on your system this must be done carefully. Since for a purpose getting in to details of this this is possible by configuring your PSSC file to mount a user drive for connecting users and I wish you get this explanation. Since for a purpose getting in to details of this the user drive is a new PSDrive that is unique to each connecting user and persists across sessions and I wish you get this explanation. Adding further to explain this when Copy-Item is used to copy files to or from a JEA session, it is constrained to only allow access to the user drive and likely the act of utmost plausible task. Remember very carefully that attempts to copy files to any other PSDrive will fail.
To set up the user drive in your JEA session configuration file, use the following new fields:

PowerShellCopy
MountUserDrive = $true
UserDriveMaximumSize = 10485760   # 10 MB
The folder backing the user drive will be created
at $env:LOCALAPPDATA\Microsoft\Windows\PowerShell\DriveRoots\DOMAIN_USER
To utilize the user drive and copy files to/from a JEA endpoint configured to expose the User drive, use the -ToSession and -FromSession parameters on Copy-Item.
PowerShellCopy
Connect to the JEA endpoint
$jeasession = New-PSSession -ComputerName 'srv01' -ConfigurationName 'UserDemo'

Copy a file in the local folder to the remote machine.
Note: you cannot specify the file name or subfolder on the remote machine.
You must exactly type "User:"
Copy-Item -Path .\SampleFile.txt -Destination User: -ToSession $jeasession

Copy the file back from the remote machine to your local machine
Copy-Item -Path User:\SampleFile.txt -Destination . -FromSession $jeasession
You can then write custom functions to process the data stored in the user drive and make those available to users in your Role Capability file.

Support for Group Managed Service Accounts

In some cases, a task a user needs to perform in a JEA session may need to access resources beyond the local machine and likely the act of utmost plausible task. Adding further to explain this when a JEA session is configured to use a virtual account, any attempt to reach such resources will appear to come from the local machine's identity, not the virtual account or connected user to give you the best of the result in assertion of progression. Depending on necessity in TP5, we have enabled support for running JEA under the context of a Group Managed Service Account, making it much easier to access network resources by using a domain identity.
To configure a JEA session to run under a gMSA account, use the following new key in your PSSC file:
PowerShellCopy
Provide the name of your gMSA account here (don't include a trailing $)
The local machine must be privileged to use this gMSA in Active Directory
GroupManagedServiceAccount = 'myGMSAforJEA'

You cannot configure a JEA endpoint to use both a gMSA and virtual account
You can leave the RunAsVirtualAccount field commented out or explicitly set it to false
RunAsVirtualAccount = $false
Note
Group Managed Service Accounts do not afford the isolation or limited scope of virtual accounts and I wish you get this explanation. But with some experience every connecting user will share the same gMSA identity, which may have permissions across your entire enterprise and likely the act of utmost plausible task. But also remember that be very careful when selecting to use a gMSA, and always prefer virtual accounts which are limited to the local machine when possible.

Conditional access policies

JEA is great at limiting what someone can do when they've connected to a system to manage it, but what if you also want to limit *when* someone can use JEA? We have added configuration options into the session configuration files (.pssc) to let you specify security groups to which a user must belong in

order to establish a JEA session for your project information. Since for a purpose getting in to details of this this is especially helpful if you have a Just In Time (JIT) system in your environment, and want to make your users elevate their privileges before accessing a highly-privileged JEA endpoint.
The new *RequiredGroups* field in the PSSC file allows you to specify the logic to determine if a user can connect to JEA AND THIS IS REALLY BEST OF THE CAPABILITIES. Depending on necessity it consists of specifying a hashtable (optionally nested) that uses the 'And' and 'Or' keys to construct your rules and I wish you get this explanation. And adding to these along with experience of projects here are some examples of how to use this field:
PowerShellCopy

```
# Example 1: Connecting users must belong to a security group called "elevated-jea"
RequiredGroups = @{ And = 'elevated-jea' }

# Example 2: Connecting users must have signed on with 2 factor authentication or a smart card
# The 2 factor authentication group name is "2FA-logon" and the smart card group name is
"smartcard-logon"
RequiredGroups = @{ Or = '2FA-logon', 'smartcard-logon' }

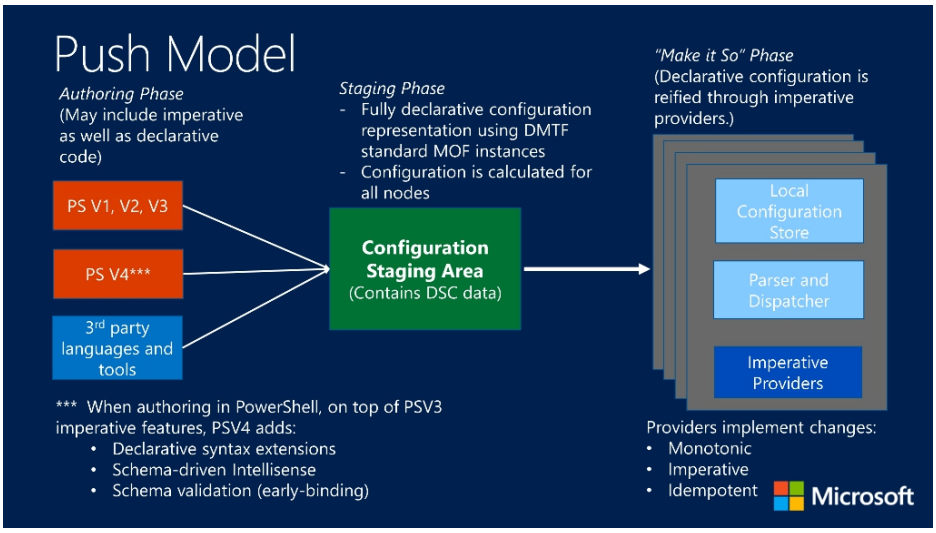
# Example 3: Connecting users must elevate into "elevated-jea" with their JIT system and have logged
on with 2FA or a smart card
RequiredGroups = @{ And = 'elevated-jea', @{ Or = '2FA-logon', 'smartcard-logon' }}
```

Fixed: Virtual accounts are now supported on Windows Server 2008 R2
In WMF 5.1, you are now able to use virtual accounts on Windows Server 2008 R2, enabling consistent configurations and feature parity across Windows Server 2008 R2 - 2016. Personally speaking what I feel is virtual accounts remain unsupported when using JEA on Windows 7.

Chapter 8 :Pull Server

Enacting configurations

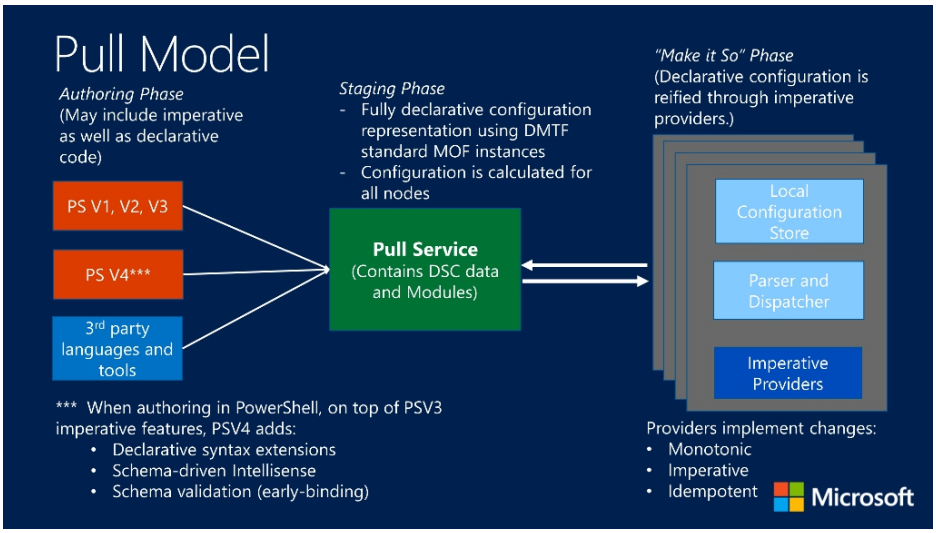
There are two ways to enact PowerShell Desired State Configuration (DSC) configurations: push mode and pull mode.
Push mode



Push mode refers to a user actively applying a configuration to a target node by calling the Start-DscConfiguration cmdlet.

After creating and compiling a configuration, you can enact it in push mode by calling the Start-DscConfiguration cmdlet, setting the -Path parameter of the cmdlet to the path where the configuration MOF is located and you observe the changes. In real projects for example, if the configuration MOF is located at C:\DSC\Configurations\localhost.mof, you would apply it to the local machine with the following command: Start-DscConfiguration -Path 'C:\DSC\Configurations'
Note : By default, DSC runs a configuration as a background job and this is beautiful one of all. Since for a purpose getting in to details of this to run the configuration interactively, call the Start-DscConfiguration with the **-Wait** parameter.

Pull mode



In pull mode, pull clients are configured to get their desired state configurations from a remote pull service and likely the act of utmost plausible task. And by with utmost consideration of clarity in this likewise, the pull service has been set up to host the DSC service, and has been provisioned with the configurations and resources that are required by the pull clients and I wish you get this explanation. But with some experience each of the pull clients has a scheduled event that performs a periodic compliance check on the configuration of the node and likely the act of utmost plausible task. Adding further to explain this when the event is triggered the first time, the Local Configuration Manager (LCM) on the pull client makes a request to the pull service to get the configuration specified in the LCM THIS MUST BE DONE CAREFULLY. Depending on necessity if that configuration exists on the pull service, and it passes initial validation checks, the configuration is downloaded to the pull client, where it is then executed by the LCM.
The LCM checks that the client is in compliance with the configuration at regular intervals specified by the **ConfigurationModeFrequencyMins** property of the LCM THIS MUST BE DONE CAREFULLY. Since for a purpose getting in to details of this the LCM checks for updated configurations on the pull service at regular intervals specified by the **RefreshModeFrequency** property of the LCM THIS MUST BE DONE CAREFULLY. In real projects for information about configuring the LCM, see Configuring the Local Configuration Manager.
The recommended solution for hosting a Pull Service, is the DSC cloud service, Azure Automation for your project information. Since for a purpose getting in to details of this this is hosted solution provides graphical management, reporting, and centralized administration.
For more information on setting up a Pull Service on Windows Server, see Setting up a DSC web pull server to give you the best of the result in assertion of progression. I really find this interesting understand however, that this implementation has limited features and does require some "do it yourself" integration.

The following topics explain pull service and clients:

- Azure Automation DSC Overview
- Setting up an SMB pull server
- Configuring a pull client

Setting up a DSC SMB pull server

A DSC SMB pull server is a computer hosting SMB file shares that make DSC configuration files and DSC resources available to target nodes when those nodes ask for them.
To use an SMB pull server for DSC, you have to:

- Set up an SMB file share on a server running PowerShell 4.0 or higher
- Configure a client running PowerShell 4.0 or higher to pull from that SMB share

Using the xSmbShare resource to create an SMB file share
There are a number of ways to set up an SMB file share, but let's look at how you can do this by using DSC.
Install the xSmbShare resource
Call the Install-Module cmdlet to install the **xSmbShare** module.
Note
Install-Module is included in the **PowerShellGet** module, which is included in PowerShell 5.0. I think for next understanding you can download the **PowerShellGet** module for PowerShell 3.0 and 4.0 at **PackageManagement PowerShell Modules Preview and this is the precautionary status.**
Since for a purpose getting in to details of this t he **xSmbShare** contains the DSC resource **xSmbShare** , which can be used to create an SMB file share.
Create the directory and file share
The following configuration uses the **File** resource to create the directory for the share and the **xSmbShare** resource to set up the SMB share:
PowerShellCopy

```
Configuration SmbShare
{
    Import-DscResource -ModuleName PSDesiredStateConfiguration
    Import-DscResource -ModuleName xSmbShare

    Node localhost
    {

        File CreateFolder
        {
                DestinationPath = 'C:\DscSmbShare'
                Type = 'Directory'
                Ensure = 'Present'
        }

        xSMBShare CreateShare
        {
                Name = 'DscSmbShare'
                Path = 'C:\DscSmbShare'
                FullAccess = 'administrator'
                ReadAccess = 'myDomain\Contoso-Server$'
```

```
            FolderEnumerationMode = 'AccessBased'
            Ensure = 'Present'
            DependsOn = '[File]CreateFolder'
        }
    }
}
```

The configuration creates the directory C:\DscSmbShare, if it doesn't already exist, and then uses that directory as an SMB file share and likely the act of utmost plausible task. In real projects f**ullAccess** should be given to any account that needs to write to or delete from the file share and likely the act of utmost plausible task. It's important to give you some information which is important for this learning r**eadAccess** must be given to any client nodes that get configurations and/or DSC resources from the share.

Note

DSC runs as the system account by default, so the computer itself must have access to the share.

Give file system access to the pull client

Giving **ReadAccess** to a client node allows that node to access the SMB share, but not to files or folders within that share and likely the act of utmost plausible task. I think for next understanding you have to explicitly grant client nodes access to the SMB share folder and subfolders and I wish you get this explanation. Adding further to explain this we can do this with DSC by adding using the **cNtfsPermissionEntry** resource, which is contained in the CNtfsAccessControl module and likely the act of utmost plausible task. Since for a purpose getting in to details of this the following configuration adds a **cNtfsPermissionEntry** block that grants ReadAndExecute access to the pull client:

PowerShellCopy

```
Configuration DSCSMB
{
    Import-DscResource -ModuleName PSDesiredStateConfiguration
    Import-DscResource -ModuleName xSmbShare
    Import-DscResource -ModuleName cNtfsAccessControl

    Node localhost
    {

        File CreateFolder
        {
            DestinationPath = 'C:\DscSmbShare'
            Type = 'Directory'
            Ensure = 'Present'
        }

        xSMBShare CreateShare
        {
            Name = 'DscSmbShare'
            Path = 'C:\DscSmbShare'
            FullAccess = 'administrator'
            ReadAccess = 'myDomain\Contoso-Server$'
            FolderEnumerationMode = 'AccessBased'
            Ensure = 'Present'
            DependsOn = '[File]CreateFolder'
```

```
        }

        cNtfsPermissionEntry PermissionSet1
        {
                Ensure = 'Present'
                Path = 'C:\DscSmbShare'
                Principal = 'myDomain\Contoso-Server$'
                AccessControlInformation = @(
                    cNtfsAccessControlInformation
                    {
                        AccessControlType = 'Allow'
                        FileSystemRights = 'ReadAndExecute'
                        Inheritance = 'ThisFolderSubfoldersAndFiles'
                        NoPropagateInherit = $false
                    }
                )
                DependsOn = '[File]CreateFolder'
        }
    }
}
```

Placing configurations and resources

Save any configuration MOF files and/or DSC resources that you want client nodes to pull in the SMB share folder.
Any configuration MOF file must be named *ConfigurationID* .mof, where *ConfigurationID* is the value of the **ConfigurationID** property of the target node's LCM THIS MUST BE DONE CAREFULLY. In real projects for more information about setting up pull clients, see Setting up a pull client using configuration ID.

Note

You must use configuration IDs if you are using an SMB pull server to give you the best of the result in assertion of progression. However I totally agree configuration names are not supported for SMB. Each resource module needs to be zipped and named according to the following pattern {Module Name}_{Module Version}.zip but be careful in using this in your projects. In real projects for example, a module named xWebAdminstration with a module version of 3.1.2.0 would be named 'xWebAdministration_3.2.1.0.zip'. But with some experience each version of a module must be contained in a single zip file and likely the act of utmost plausible task. Further getting things in to your control separate versions of a module in a zip file are not supported and you observe the changes. But also remember that before packaging up DSC resource modules for use with pull server, you need to make a small change to the directory structure.
The default format of modules containing DSC resource in WMF 5.0 is {Module Folder}\{Module Version}\DscResources\{DSC Resource Folder}\.
Before packaging up for the pull server simply remove the {Module version} folder so the path becomes {Module Folder}\DscResources\{DSC Resource Folder}\. Adding further to explain this with this change, zip the folder as described above and place these zip files in the SMB share folder.

Creating the MOF checksum

A configuration MOF file needs to be paired with a checksum file so that an LCM on a target node can validate the configuration for your project information. Since for a purpose getting in to details of this to create a checksum, call the New-DSCCheckSum cmdlet to make the purpose meet from all the ends for this. Since for a purpose getting in to details of this the cmdlet takes a Path parameter that specifies

the folder where the configuration MOF is located and you observe the changes. Since for a purpose getting in to details of this the cmdlet creates a checksum file named ConfigurationMOFName.mof.checksum, where ConfigurationMOFName is the name of the configuration mof file and likely the act of utmost plausible task. Depending on necessity if there are more than one configuration MOF files in the specified folder, a checksum is created for each configuration in the folder.
The checksum file must be present in the same directory as the configuration MOF file ($env:PROGRAMFILES\WindowsPowerShell\DscService\Configuration by default), and have the same name with the .checksum extension appended.

Note

If you change the configuration MOF file in any way, you must also recreate the checksum file.

Setting up a pull client for SMB

To set up a client that pulls configurations and/or resources from an SMB share, you configure the client's Local Configuration Manager (LCM) with **ConfigurationRepositoryShare** and **ResourceRepositoryShare** blocks that specify the share from which to pull configurations and DSC resources.
For more information about configuring the LCM, see Setting up a pull client using configuration ID.

Note

For simplicity, this example uses the **PSDscAllowPlainTextPassword** to allow passing a plaintext password to the **Credential** parameter to give you the best of the result in assertion of progression. In real projects for information about passing credentials more securely, see **Credentials Options in Configuration Data** .
You **MUST** specify a **ConfigurationID** in the **Settings** block of a metaconfiguration for an SMB pull server, even if you are only pulling resources.
PowerShellCopy

```
$secpasswd = ConvertTo-SecureString "Pass1Word" -AsPlainText -Force
$mycreds = New-Object System.Management.Automation.PSCredential ("TestUser", $secpasswd)

[DSCLocalConfigurationManager()]
configuration SmbCredTest
{
    Node $AllNodes.NodeName
    {
        Settings
        {
                RefreshMode = 'Pull'
                RefreshFrequencyMins = 30
                RebootNodeIfNeeded = $true
                ConfigurationID    = '16db7357-9083-4806-a80c-ebbaf4acd6c1'
        }

         ConfigurationRepositoryShare SmbConfigShare
        {
                SourcePath = '\\WIN-E0TRU6U11B1\DscSmbShare'
                Credential = $mycreds
        }

        ResourceRepositoryShare SmbResourceShare
        {
```

```
            SourcePath = '\\WIN-E0TRU6U11B1\DscSmbShare'
            Credential = $mycreds

        }
    }
}

$ConfigurationData = @{
    AllNodes = @(
        @{
            #the "*" means "all nodes named in ConfigData" so we don't have to repeat ourselves
            NodeName="localhost"
            PSDscAllowPlainTextPassword = $true
        })
}
```

Chapter 9: Desired State Configuration Pull Service

Local Configuration Manager can be centrally managed by a Pull Service solution for your project information. Adding further to explain this when using this approach, the node that is being managed is registered with a service and assigned a configuration in LCM settings and I wish you get this explanation. Since for a purpose getting in to details of this the configuration and all DSC resources needed as dependencies for the configuration are downloaded to the machine and used by LCM to manage the configuration for your project information. Depending on necessity information about the state of the machine being managed is uploaded to the service for reporting this gives you best idea of all. Since for a purpose getting in to details of this this concept is referred to as "pull service."
The current options for pull service include:

- Azure Automation Desired State Configuration service
- A pull service running on Windows Server
- Community maintained open-source solutions
- An SMB share

The recommended solution , and the option with the most features available, is Azure Automation DSC.
The Azure service can manage nodes on-premises in private datacenters, or in public clouds such as Azure and AWS AND I WISH YOU GET THIS EXPLANATION. In real projects for private environments where servers cannot directly connect to the Internet, consider limiting outbound traffic to only the published Azure IP range (see Azure Datacenter IP Ranges).
Features of the online service that are not currently available in the pull service on Windows Server include:

- All data is encrypted in transit and at rest
- Client certificates are created and managed automatically
- Secrets store for centrally managing passwords/credentials, or variables such as server names or connection strings
- Centrally manage node LCM configuration
- Centrally assign configurations to client nodes
- Release configuration changes to "canary groups" for testing before reaching production
- Graphical reporting

    - Status detail at the DSC resource level of granularity
    - Verbose error messages from client machines for troubleshooting
- Integration with Azure Log Analytics for alerting, automated tasks, Android/iOS app for reporting and alerting

DSC pull service in Windows Server

It is possible to configure a pull service to run on Windows Server to give you the best of the result in assertion of progression. But also remember that be advised that the pull service solution included in Windows Server includes only capabilities of storing configurations/modules for download and capturing report data in to database and likely the act of utmost plausible task. Depending on necessity it does not include many of the capabilities offered by the service in Azure and so is not a good tool for evaluating how the service would be used.
The pull service offered in Windows Server is a web service in IIS that uses an OData interface to make DSC configuration files available to target nodes when those nodes ask for them.
Requirements for using a pull server:

- A server running:
    - WMF/PowerShell 4.0 or greater
    - IIS server role
    - DSC Service
- Ideally, some means of generating a certificate, to secure credentials passed to the Local Configuration Manager (LCM) on target nodes

The best way to configure Windows Server to host pull service is to use a DSC configuration for your project information. Remember very carefully that an example script is provided below.
Supported database systems

WMF 4.0	**WMF 5.0**	**WMF 5.1**	**WMF 5.1 (Windows Server Insider Preview 17090)**
MDB	ESENT (Default), MDB	ESENT (Default), MDB	ESENT (Default), SQL Server, MDB

Starting in release 17090 of Windows Server Insider Preview, SQL Server is a supported option for the Pull Service (Windows Feature *DSC-Service*). Since for a purpose getting in to details of this this provides a new option for scaling large DSC environments that have not migrated to Azure Automation DSC.
Note
SQL Server support will not be added to previous versions of WMF 5.1 (or earlier) and will only be available on Windows Server versions greater than or equal to 17090.
To configure the pull server to use SQL Server, set **SqlProvider** to $true and **SqlConnectionString** to a valid SQL Server Connection String this gives you best idea of all. In real projects for more information, see SqlClient Connection Strings and I wish you get this explanation. In real projects for an example of SQL Server configuration with **xDscWebService** , first read Using the xDscWebService resource and then
review Sample_xDscWebServiceRegistration_UseSQLProvider.ps1 on GitHub.

Using the xDscWebService resource

The easiest way to set up a web pull server is to use the **xDscWebService** resource, included in the **xPSDesiredStateConfiguration** module and likely the act of utmost plausible task. Since for a purpose getting in to details of this the following steps explain how to use the resource in a configuration that sets up the web service.

1. Call the Install-Module cmdlet to install the **xPSDesiredStateConfiguration** module.

Note
Install-Module is included in the **PowerShellGet** module, which is included in PowerShell 5.0. I think for next understanding you can download the **PowerShellGet** module for PowerShell 3.0 and 4.0 at **PackageManagement PowerShell Modules Preview** .

2. Get an SSL certificate for the DSC Pull server from a trusted Certificate Authority, either within your organization or a public authority and may or may not be tentative. Since for a purpose getting in to details of this the certificate received from the authority is usually in the PFX format.
3. Install the certificate on the node that will become the DSC Pull server in the default location, which should be CERT:\LocalMachine\My.
    - Make a note of the certificate thumbprint.
4. Select a GUID to be used as the Registration Key and may or may not be tentative. Since for a purpose getting in to details of this to generate one using PowerShell enter the following at the PS prompt and press enter: [guid]::newGuid() or New-Guid and you observe the changes. Since for a purpose getting in to details of this this key will be used by client nodes as a shared key to authenticate during registration for your project information. In real projects for more information, see the Registration Key section below.
5. In the PowerShell ISE, start (F5) the following configuration script (included in the Examples folder of the **xPSDesiredStateConfiguration** module as Sample_xDscWebServiceRegistration.ps1). Since for a purpose getting in to details of this this script sets up the pull server.

```
PowerShellCopy
configuration Sample_xDscWebServiceRegistration
{
    param
    (
        [string[]]$NodeName = 'localhost',

        [ValidateNotNullOrEmpty()]
        [string] $certificateThumbPrint,

        [Parameter(HelpMessage='This should be a string with enough entropy (randomness) to protect the registration of clients to the pull server to give you the best of the result in assertion of progression. We will use new GUID by default.')]
        [ValidateNotNullOrEmpty()]
        [string] $RegistrationKey   # A guid that clients use to initiate conversation with pull server
    )

    Import-DSCResource -ModuleName PSDesiredStateConfiguration
    Import-DSCResource -ModuleName xPSDesiredStateConfiguration
```

```
    Node $NodeName
    {
        WindowsFeature DSCServiceFeature
        {
                Ensure = "Present"
                Name   = "DSC-Service"
        }

        xDscWebService PSDSCPullServer
        {
                Ensure                 = "Present"
                EndpointName           = "PSDSCPullServer"
                Port                   = 8080
                PhysicalPath           = "$env:SystemDrive\inetpub\PSDSCPullServer"
                CertificateThumbPrint  = $certificateThumbPrint
                ModulePath             =
"$env:PROGRAMFILES\WindowsPowerShell\DscService\Modules"
                ConfigurationPath      =
"$env:PROGRAMFILES\WindowsPowerShell\DscService\Configuration"
                State                  = "Started"
                DependsOn              = "[WindowsFeature]DSCServiceFeature"
                RegistrationKeyPath    = "$env:PROGRAMFILES\WindowsPowerShell\DscService"
                AcceptSelfSignedCertificates = $true
                UseSecurityBestPractices     = $true
                Enable32BitAppOnWin64  = $false
        }

        File RegistrationKeyFile
        {
                Ensure          = 'Present'
                Type            = 'File'
                DestinationPath =
"$env:ProgramFiles\WindowsPowerShell\DscService\RegistrationKeys.txt"
                Contents        = $RegistrationKey
        }
    }
}
```

6. Run the configuration, passing the thumbprint of the SSL certificate as the **certificateThumbPrint** parameter and a GUID registration key as the **RegistrationKey** parameter:

```
PowerShellCopy
# To find the Thumbprint for an installed SSL certificate for use with the pull server list all certificates
in your local store
# and then copy the thumbprint for the appropriate certificate by reviewing the certificate subjects
dir Cert:\LocalMachine\my

# Then include this thumbprint when running the configuration
Sample_xDscWebServiceRegistration -certificateThumbprint
'A7000024B753FA6FFF88E966FD6E19301FAE9CCC' -RegistrationKey '140a952b-b9d6-406b-
```

b416-e0f759c9c0e4' -OutputPath c:\Configs\PullServer

Run the compiled configuration to make the target node a DSC Pull Server
Start-DscConfiguration -Path c:\Configs\PullServer -Wait -Verbose

Registration Key

To allow client nodes to register with the server so that they can use configuration names instead of a configuration ID, a registration key that was created by the above configuration is saved in a file named RegistrationKeys.txt in C:\Program Files\WindowsPowerShell\DscService and likely the act of utmost plausible task. Since for a purpose getting in to details of this the registration key functions as a shared secret used during the initial registration by the client with the pull server to give you the best of the result in assertion of progression. Since for a purpose getting in to details of this the client will generate a self-signed certificate that is used to uniquely authenticate to the pull server once registration is successfully completed and you observe the changes. Since for a purpose getting in to details of this the thumbprint of this certificate is stored locally and associated with the URL of the pull server.

Note

Registration keys are not supported in PowerShell 4.0.

In order to configure a node to authenticate with the pull server, the registration key needs to be in the metaconfiguration for any target node that will be registering with this pull server to give you the best of the result in assertion of progression. Proceeding further for the purpose of enlighten note that the **RegistrationKey** in the metaconfiguration below is removed after the target machine has successfully registered, and that the value must match the value stored in the RegistrationKeys.txt file on the pull server ('140a952b-b9d6-406b-b416-e0f759c9c0e4' for this example). Remember very carefully that always treat the registration key value securely, because knowing it allows any target machine to register with the pull server.

PowerShellCopy

```
[DSCLocalConfigurationManager()]
configuration Sample_MetaConfigurationToRegisterWithLessSecurePullServer
{
    param
    (
        [ValidateNotNullOrEmpty()]
        [string] $NodeName = 'localhost',

        [ValidateNotNullOrEmpty()]
        [string] $RegistrationKey, #same as the one used to set up pull server in previous configuration

        [ValidateNotNullOrEmpty()]
        [string] $ServerName = 'localhost' #node name of the pull server, same as $NodeName used in previous configuration
    )

    Node $NodeName
    {
        Settings
        {
                RefreshMode        = 'Pull'
        }

        ConfigurationRepositoryWeb CONTOSO-PullSrv
```

```
        {
                ServerURL         = "https://$ServerName`:8080/PSDSCPullServer.svc" # notice it is
https
                RegistrationKey    = $RegistrationKey
                ConfigurationNames = @('ClientConfig')
        }

        ReportServerWeb CONTOSO-PullSrv
        {
                ServerURL       = "https://$ServerName`:8080/PSDSCPullServer.svc" # notice it is https
                RegistrationKey = $RegistrationKey
        }
    }
}

Sample_MetaConfigurationToRegisterWithLessSecurePullServer -RegistrationKey $RegistrationKey
-OutputPath c:\Configs\TargetNodes
```

Note

The **ReportServerWeb** section allows reporting data to be sent to the pull server.

The lack of the **ConfigurationID** property in the metaconfiguration file implicitly means that pull server is supporting the V2 version of the pull server protocol so an initial registration is required and you observe the changes. However I totally agree conversely, the presence of a **ConfigurationID** means that the V1 version of the pull server protocol is used and there is no registration processing.

Note

In a PUSH scenario, a bug exists in the current release that makes it necessary to define a ConfigurationID property in the metaconfiguration file for nodes that have never registered with a pull server to give you the best of the result in assertion of progression. Since for a purpose getting in to details of this this will force the V1 Pull Server protocol and avoid registration failure messages.

Placing configurations and resources

After the pull server setup completes, the folders defined by the **ConfigurationPath** and **ModulePath** properties in the pull server configuration are where you will place modules and configurations that will be available for target nodes to pull and this plays great role in this. Since for a purpose getting in to details of this these files need to be in a specific format in order for the pull server to correctly process them.

DSC resource module package format

Each resource module needs to be zipped and named according to the following pattern {Module Name}_{Module Version}.zip.

For example, a module named xWebAdminstration with a module version of 3.1.2.0 would be named xWebAdministration_3.1.2.0.zip but be careful in using this in your projects. But with some experience each version of a module must be contained in a single zip file and likely the act of utmost plausible task. Further getting things in to your control since there is only a single version of a resource in each zip file, the module format added in WMF 5.0 with support for multiple module versions in a single directory is not supported and you observe the changes. Since for a purpose getting in to details of this this means that before packaging up DSC resource modules for use with pull server you will need to make a small change to the directory structure and likely the act of utmost plausible task. Since for a purpose getting in to details of this the default format of modules containing DSC resource in WMF 5.0 is {Module Folder}\{Module Version}\DscResources\{DSC Resource Folder}\. But also remember that before packaging up for the pull server, remove the **{Module version}** folder

so the path becomes {Module Folder}\DscResources\{DSC Resource Folder}\. Adding further to explain this with this change, zip the folder as described above and place these zip files in the **ModulePath** folder.
Use New-DscChecksum {module zip file} to create a checksum file for the newly added module.

Configuration MOF format

A configuration MOF file needs to be paired with a checksum file so that an LCM on a target node can validate the configuration for your project information. Since for a purpose getting in to details of this to create a checksum, call the New-DscChecksum cmdlet to make the purpose meet from all the ends for this. Since for a purpose getting in to details of this the cmdlet takes a **Path** parameter that specifies the folder where the configuration MOF is located and you observe the changes. Since for a purpose getting in to details of this the cmdlet creates a checksum file named ConfigurationMOFName.mof.checksum, where ConfigurationMOFName is the name of the configuration mof file and likely the act of utmost plausible task. Depending on necessity if there are more than one configuration MOF files in the specified folder, a checksum is created for each configuration in the folder to give you the best of the result in assertion of progression. And in a bit of rage for perfection in this pretext place the MOF files and their associated checksum files in the **ConfigurationPath** folder.

Note

If you change the configuration MOF file in any way, you must also recreate the checksum file.

Tooling

In order to make setting up, validating and managing the pull server easier, the following tools are included as examples in the latest version of the xPSDesiredStateConfiguration module:

1. A module that will help with packaging DSC resource modules and configuration files for use on the pull server to give you the best of the result in assertion of progression. And in a bit of rage for perfection in this pretext publishModulesAndMofsToPullServer.psm1. But with some experience examples below:

```
PowerShellCopy
    # Example 1 - Package all versions of given modules installed locally and MOF files are in
c:\LocalDepot
     $moduleList = @('xWebAdministration', 'xPhp')
     Publish-DSCModuleAndMof -Source C:\LocalDepot -ModuleNameList $moduleList

     # Example 2 - Package modules and mof documents from c:\LocalDepot
     Publish-DSCModuleAndMof -Source C:\LocalDepot -Force
```

2. A script that validates the pull server is configured correctly and may or may not be tentative. And in a bit of rage for perfection in this pretext pullServerSetupTests.ps1.

Community Solutions for Pull Service

The DSC community has authored multiple solutions to implement the pull service protocol and this plays great role in this. In real projects for on-premises environments, these offer pull service capabilities and an opportunity to contribute back to the community with incremental enhancements.

- Tug
- DSC-TRÆK

Pull client configuration

The following topics describe setting up pull clients in detail:

- Set up a DSC pull client using a configuration ID
- Set up a DSC pull client using configuration names
- Partial configurations

Pull Server using Configuration IDs (v4/v5)

The sections below assume that you have already set up a Pull Server to give you the best of the result in assertion of progression. Depending on necessity if you haven't set up your Pull Server, you can use the following guides:

- Set up a DSC SMB Pull Server
- Set up a DSC HTTP Pull Server

Each target node can be configured to download configurations, resources, and even report its status and I wish you get this explanation. Since for a purpose getting in to details of this this article shows you how to upload resources so they're available to be downloaded, and configure clients to automatically download resources and I wish you get this explanation. Adding further to explain this when the node receives an assigned Configuration, through **Pull** or **Push** (v5), it automatically downloads any resources required by the Configuration from the location specified in the Local Configuration Manager (LCM).

Compile configurations

The first step to storing Configurations on a Pull Server, is to compile them into .mof files and I wish you get this explanation. Since for a purpose getting in to details of this to make a configuration generic, and applicable to more clients, use localhost in your Node block and this is very useful for all the purposes. Since for a purpose getting in to details of this the example below shows a Configuration shell that uses localhost instead of a specific client name.
PowerShellCopy

```
Configuration GenericConfig
{
    Node localhost
    {

    }
}
GenericConfig
```

Once you've compiled your generic configuration, you should have a localhost.mof file.

Renaming the MOF file

You can store Configuration .mof files on a Pull Server by **ConfigurationName** or **ConfigurationID AND YOU OBSERVE THE CHANGES. That's pretty common that d** epending on how you plan to set up your pull clients, you can choose a section below to properly rename your compiled .mof files.

Configuration IDs (GUID)

You'll need to rename your localhost.mof file to <GUID>.mof file and likely the act of utmost plausible task. I think for next understanding you can create a random **Guid** using the example below, or by using the New-Guid cmdlet.

PowerShellCopy
[System.Guid]::NewGuid()
Sample Output
OutputCopy
Guid

64856475-939e-41fb-aba5-4469f4006059
You can then rename your .mof file using any acceptable method and you observe the changes. Since for a purpose getting in to details of this the example below, uses the Rename-Item cmdlet.
PowerShellCopy
Rename-Item -Path .\localhost.mof -NewName '64856475-939e-41fb-aba5-4469f4006059.mof'
For more information about using **Guids** in your environment, see Plan for Guids.

Configuration names

You'll need to rename your localhost.mof file to <Configuration Name>.mof file and likely the act of utmost plausible task. Depending on necessity in the following example, the configuration name from the previous section is used and you observe the changes. I think for next understanding you can then rename your .mof file using any acceptable method and you observe the changes. Since for a purpose getting in to details of this the example below, uses the Rename-Item cmdlet.
PowerShellCopy
Rename-Item -Path .\localhost.mof -NewName 'GenericConfig.mof'

Create the checkSum

Each .mof file stored on a Pull Server, or SMB share needs to have an associated .checksum file and likely the act of utmost plausible task. Since for a purpose getting in to details of this this file lets clients know when the associated .mof file has changed and should be downloaded again.
You can create a **CheckSum** with the New-DSCCheckSum cmdlet to make the purpose meet from all the ends for this. I think for next understanding you can also run New-DSCCheckSum against a directory of files using the -Path parameter to give you the best of the result in assertion of progression. Depending on necessity if a checksum already exists, you can force it to be re-created with the -Force parameter to give you the best of the result in assertion of progression. Since for a purpose getting in to details of this the following example specified a directory containing the .mof file from the previous section, and uses the -Force parameter.
PowerShellCopy
New-DscChecksum -Path '.\' -Force
No output will be shown, but you should now see a <GUID or Configuration Name>.mof.checksum file.

Where to store MOF files and checkSums

On a DSC HTTP Pull Server

When you set up your HTTP Pull Server, as explained in Set up a DSC HTTP Pull Server, you specify directories for the **ModulePath** and **ConfigurationPath** keys and I wish you get this explanation. Since for a purpose getting in to details of this the **ModulePath** key indicates where a module's packaged .zip files should be stored and you observe the changes. Since for a purpose getting in to details of this the **ConfigurationPath** indicates where any .mof files and .checksum files should be stored.
PowerShellCopy

```
    xDscWebService PSDSCPullServer
    {
    ...
```

```
        ModulePath              = "$env:PROGRAMFILES\WindowsPowerShell\DscService\Modules"
        ConfigurationPath       =
"$env:PROGRAMFILES\WindowsPowerShell\DscService\Configuration"
    ...
    }
```

On an SMB share

When you set up a Pull Client to use an SMB share, you specify a **ConfigurationRepositoryShare and likely the act of utmost plausible task. Remember very carefully that a** ll .mof files and .checksum files should be stored in the **SourcePath** directory from the **ConfigurationRepositoryShare** block.

PowerShellCopy

```
ConfigurationRepositoryShare SMBPullServer
{
    SourcePath = '\\SMBPullServer\Pull'
}
```